PPSE Firmware

1b0c3c6

Generated by Doxygen 1.9.8

1 RP2040 GPS & Environment Logger 1

1.1 Core Features . . . 1

1.2 Risorse e Documentazione . . . 1

1.3 Struttura del Repository . . . 1

1.3.1 Contributi e Sviluppo . . . 1

2 Data Structure Index 3

2.1 Data Structures . . . 3

3 File Index 5

3.1 File List . . . 5

4 Data Structure Documentation 7

4.1 parsed_nmea Struct Reference . . . 7

4.1.1 Detailed Description . . . 7

4.1.2 Field Documentation . . . 7

4.1.2.1 parsed_gbs . . . 7

4.1.2.2 parsed_gga . . . 7

4.1.2.3 parsed_gll . . . 7

4.1.2.4 parsed_gsa . . . 8

4.1.2.5 parsed_gst . . . 8

4.1.2.6 parsed_gsv . . . 8

4.1.2.7 parsed_rmc . . . 8

4.1.2.8 parsed_vtg . . . 8

4.1.2.9 parsed_zda . . . 8

4.2 WeatherState Struct Reference . . . 8

4.2.1 Detailed Description . . . 9

4.2.2 Field Documentation . . . 9

4.2.2.1 city . . . 9

4.2.2.2 curHumidity . . . 9

4.2.2.3 curTempC . . . 9

4.2.2.4 curValid . . . 9

4.2.2.5 curWcode . . . 9

4.2.2.6 curWindKmh . . . 9

4.2.2.7 fcValid . . . 9

4.2.2.8 lastUpdateMs . . . 9

4.2.2.9 recvMask . . . 10

4.2.2.10 tempC . . . 10

4.2.2.11 wcode . . . 10

4.2.2.12 weekday0 . . . 10

5 File Documentation 11

5.1 firm/include/buttons.h File Reference . . . 11

5.1.1 Detailed Description . . . 12

5.1.2 Macro Definition Documentation . . . 13
5.1.2.1 SW_DOWN . . . 13
5.1.2.2 SW_LEFT . . . 13
5.1.2.3 SW_RIGHT . . . 13
5.1.2.4 SW_UP . . . 13
5.1.3 Enumeration Type Documentation . . . 13
5.1.3.1 KeyEvent . . . 13
5.1.4 Function Documentation . . . 13
5.1.4.1 buttons_callback() . . . 13
5.1.4.2 buttonsInit() . . . 14
5.1.4.3 buttonsUpdate() . . . 14
5.1.4.4 checkLeftRightCombo() . . . 14
5.1.4.5 debounce_callback() . . . 14
5.1.4.6 onLeftRightCombo() . . . 15
5.1.5 Variable Documentation . . . 15
5.1.5.1 ambientDisplay . . . 15
5.1.5.2 COMBO_MS . . . 15
5.1.5.3 DISPLAY_TIMEOUT_MS . . . 15
5.1.5.4 down_press . . . 15
5.1.5.5 inSubmenu . . . 15
5.1.5.6 lastActivity . . . 15
5.1.5.7 ledsNumber . . . 15
5.1.5.8 left_press . . . 16
5.1.5.9 menuIndex . . . 16
5.1.5.10 menuOpen . . . 16
5.1.5.11 right_press . . . 16
5.1.5.12 tLeft . . . 16
5.1.5.13 tRight . . . 16
5.1.5.14 up_press . . . 16
5.2 buttons.h . . . 16
5.3 firm/include/buzzer.h File Reference . . . 17
5.3.1 Detailed Description . . . 18
5.3.2 Macro Definition Documentation . . . 18
5.3.2.1 BUZZER . . . 18
5.3.3 Function Documentation . . . 18
5.3.3.1 buzzerBeep() . . . 18
5.3.3.2 buzzerInit() . . . 19
5.4 buzzer.h . . . 19
5.5 firm/include/display_ui.h File Reference . . . 19
5.5.1 Detailed Description . . . 21
5.5.2 Macro Definition Documentation . . . 21
5.5.2.1 DISPLAY_SCL_I2C_PIN . . . 21

5.5.2.2 DISPLAY_SDA_I2C_PIN . . . 21
5.5.2.3 OLED_RESET . . . 21
5.5.2.4 SCREEN_ADDRESS . . . 22
5.5.2.5 SCREEN_HEIGHT . . . 22
5.5.2.6 SCREEN_WIDTH . . . 22
5.5.3 Function Documentation . . . 22
5.5.3.1 displayInit() . . . 22
5.5.3.2 drawGPSScreen() . . . 22
5.5.3.3 drawHomeScreen() . . . 22
5.5.3.4 drawInfoScreen() . . . 22
5.5.3.5 drawMenu() . . . 22
5.5.3.6 drawMenu_evi() . . . 23
5.5.3.7 drawMeteoScreen() . . . 23
5.5.3.8 drawSelected() . . . 23
5.5.3.9 drawSettingsScreen() . . . 23
5.5.3.10 drawSystemScreen() . . . 23
5.5.3.11 setBrightness() . . . 23
5.5.3.12 updateDisplayTimeout() . . . 23
5.5.4 Variable Documentation . . . 24
5.5.4.1 gpsData . . . 24
5.5.4.2 gpsDayIndex . . . 24
5.5.4.3 gpsHourIndex . . . 24
5.5.4.4 menuItems . . . 24
5.5.4.5 menuLength . . . 24
5.5.4.6 ScreenStyle . . . 24
5.6 display_ui.h . . . 24
5.7 firm/include/filesystem.h File Reference . . . 25
5.7.1 Detailed Description . . . 26
5.7.2 Function Documentation . . . 27
5.7.2.1 FatFS_Setup() . . . 27
5.7.2.2 FatFSUSB_Setup() . . . 27
5.7.2.3 mountable() . . . 27
5.7.2.4 plug() . . . 27
5.7.2.5 sync_files() . . . 27
5.7.2.6 unplug() . . . 28
5.7.3 Variable Documentation . . . 28
5.7.3.1 driveConnected . . . 28
5.7.3.2 inPrinting . . . 28
5.7.3.3 updated . . . 28
5.8 filesystem.h . . . 28
5.9 firm/include/gps.h File Reference . . . 29
5.9.1 Detailed Description . . . 31

5.9.2 Macro Definition Documentation . . . 31
5.9.2.1 GPS_ACQUIRE_MAX_TRIES . . . 31
5.9.2.2 GPS_EN_PIN . . . 31
5.9.2.3 GPS_LAST_PATH . . . 31
5.9.2.4 GPS_LOG_PATH . . . 31
5.9.2.5 GPS_MAX_LINES . . . 31
5.9.2.6 GPS_PPS_PIN . . . 31
5.9.2.7 GPS_SYNC_ACK . . . 32
5.9.2.8 GPS_SYNC_REQ . . . 32
5.9.2.9 GPS_SYNC_TIMEOUT_MSEC . . . 32
5.9.2.10 GPS_WRITE_FREQ . . . 32
5.9.2.11 GPSBAUD . . . 32
5.9.2.12 LED_ALIVE . . . 32
5.9.2.13 MINMEA_MAX_SENTENCE_LENGTH . . . 32
5.9.2.14 UART0_RX_PIN . . . 32
5.9.2.15 UART0_TX_PIN . . . 32
5.9.2.16 UART1_RX_PIN . . . 32
5.9.2.17 UART1_TX_PIN . . . 33
5.9.3 Function Documentation . . . 33
5.9.3.1 check_GPS_sync_req() . . . 33
5.9.3.2 GetDate_and_Time() . . . 33
5.9.3.3 GetPosition_and_Satellites() . . . 33
5.9.3.4 GPS_sync() . . . 33
5.9.3.5 gpsAcquire() . . . 33
5.9.3.6 gpsInit() . . . 33
5.9.3.7 nmea_gps_parse() . . . 34
5.9.3.8 print_NMEA_gga() . . . 34
5.9.3.9 print_NMEA_gst() . . . 34
5.9.3.10 print_NMEA_gsv() . . . 34
5.9.3.11 print_NMEA_rmc() . . . 34
5.9.3.12 print_NMEA_vtg() . . . 34
5.9.3.13 print_NMEA_zda() . . . 34
5.9.3.14 save_gps_last() . . . 34
5.9.3.15 save_gps_log20() . . . 35
5.9.4 Variable Documentation . . . 35
5.9.4.1 gpsDirty . . . 35
5.9.4.2 lastUiGpsMs . . . 35
5.10 gps.h . . . 35
5.11 firm/include/leds.h File Reference . . . 36
5.11.1 Detailed Description . . . 37
5.11.2 Macro Definition Documentation . . . 37
5.11.2.1 COLOR_ORDER . . . 37

5.11.2.2 LED_PIN . . . 38
5.11.2.3 LED_TYPE . . . 38
5.11.2.4 NUM_LEDS . . . 38
5.11.3 Function Documentation . . . 38
5.11.3.1 ledsInactivityAnimation() . . . 38
5.11.3.2 ledsInit() . . . 38
5.11.3.3 ledsNavigationAnimation() . . . 38
5.11.3.4 ledsShowInitAnimation() . . . 38
5.11.3.5 ledsToggleAnimation() . . . 38
5.12 leds.h . . . 39
5.13 firm/include/temp.h File Reference . . . 39
5.13.1 Detailed Description . . . 40
5.13.2 Macro Definition Documentation . . . 40
5.13.2.1 ADC_BITS . . . 40
5.13.2.2 ADC_MAX_VAL . . . 40
5.13.2.3 TEMP_PIN . . . 41
5.13.3 Function Documentation . . . 41
5.13.3.1 readTemp() . . . 41
5.13.3.2 TempInit() . . . 41
5.14 temp.h . . . 41
5.15 firm/include/weather.h File Reference . . . 41
5.15.1 Detailed Description . . . 42
5.15.2 Function Documentation . . . 43
5.15.2.1 Weather_CodeToShortText() . . . 43
5.15.2.2 Weather_HandleLine() . . . 43
5.15.2.3 Weather_Init() . . . 43
5.15.2.4 Weather_WeekdayName() . . . 43
5.15.3 Variable Documentation . . . 44
5.15.3.1 wx . . . 44
5.16 weather.h . . . 44
5.17 buttons.cpp . . . 44
5.18 buzzer.cpp . . . 49
5.19 display_ui.cpp . . . 49
5.20 filesystem.cpp . . . 53
5.21 gps.cpp . . . 54
5.22 leds.cpp . . . 59
5.23 temp.cpp . . . 59
5.24 weather.cpp . . . 60

Chapter 1

RP2040 GPS & Environment Logger

Firmware avanzato per **RP2040** dedicato al logging geospaziale, monitoraggio ambientale e telemetria in tempo reale.

1.1 Core Features

- **GPS Integration:** Sincronizzazione oraria u-blox e logging coordinate NMEA.
- **Hybrid Storage:** Gestione dati su Flash interna via **LittleFS** e USB Mass Storage.
- **UI & UX:** Interfaccia grafica su OLED SSD1306, LED RGB adattivi e feedback sonoro.
- **Power Management:** Monitoraggio dei bus di alimentazione per operazioni standalone.

1.2 Risorse e Documentazione

Tutte le informazioni dettagliate sono centralizzate nelle seguenti risorse esterne, aggiornate automaticamente tramite CI/CD:

- `Project Wiki` – **Guida Rapida**, comandi di build (`pio`, `make`), manuale utente e scorciatoie.
- `Academic Report (PDF)` – Analisi tecnica, schemi elettrici e architettura del sistema.
- `API Reference` – Documentazione tecnica Doxygen per classi e driver.

1.3 Struttura del Repository

- `src/`: Logica applicativa e task core.
- `lib/`: Driver hardware (GPS, sensori, display).
- `report/`: Sorgenti LaTeX della documentazione accademica.
- `site/`: Dashboard statica per il portale web del progetto.

1.3.1 Contributi e Sviluppo

Se desideri contribuire o compilare il firmware localmente, consulta la sezione **Sviluppo** nella `Wiki del progetto`. Per segnalazioni di bug, apri una **Issue** o una **Pull Request**.

Chapter 2

Data Structure Index

2.1 Data Structures

Here are the data structures with brief descriptions:

parsed_nmea
Container globale per le sentenze NMEA parsate 7
WeatherState
Contenitore globale per lo stato meteorologico 8

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

firm/include/buttons.h
Definizioni e prototipi per la gestione degli input utente e della UI 11
firm/include/buzzer.h
Definizioni e prototipi per il controllo del feedback sonoro 17
firm/include/display_ui.h
Definizioni hardware e prototipi per l'interfaccia utente OLED 19
firm/include/filesystem.h
Interfaccia per la gestione del filesystem FatFS e l'integrazione USB Mass Storage 25
firm/include/gps.h
Definizioni, macro e strutture per la gestione della telemetria GPS 29
firm/include/leds.h
Definizioni hardware e prototipi per la gestione dei LED RGB 36
firm/include/temp.h
Definizioni hardware e prototipi per la lettura della temperatura 39
firm/include/weather.h
Definizioni della struttura dati e prototipi per il modulo Meteo 41
firm/src/buttons.cpp 44
firm/src/buzzer.cpp 49
firm/src/display_ui.cpp 49
firm/src/filesystem.cpp 53
firm/src/gps.cpp 54
firm/src/leds.cpp 59
firm/src/temp.cpp 59
firm/src/weather.cpp 60

Chapter 4

Data Structure Documentation

4.1 parsed_nmea Struct Reference

Container globale per le sentenze NMEA parsate.
`#include <gps.h>`

Data Fields

- struct minmea_sentence_gbs parsed_gbs
- struct minmea_sentence_rmc parsed_rmc
- struct minmea_sentence_gga parsed_gga
- struct minmea_sentence_gsa parsed_gsa
- struct minmea_sentence_gll parsed_gll
- struct minmea_sentence_gst parsed_gst
- struct minmea_sentence_gsv parsed_gsv
- struct minmea_sentence_vtg parsed_vtg
- struct minmea_sentence_zda parsed_zda

4.1.1 Detailed Description

Container globale per le sentenze NMEA parsate.

- Questa struttura aggrega i diversi tipi di messaggi NMEA gestiti dalla libreria `minmea`. Viene aggiornata ogni volta che una nuova stringa valida viene ricevuta dalla seriale.

Definition at line 75 of file gps.h.

4.1.2 Field Documentation

4.1.2.1 parsed_gbs

`struct minmea_sentence_gbs parsed_nmea::parsed_gbs`
Errori satellitari.
Definition at line 76 of file gps.h.

4.1.2.2 parsed_gga

`struct minmea_sentence_gga parsed_nmea::parsed_gga`
Fix di precisione e altitudine.
Definition at line 78 of file gps.h.

4.1.2.3 parsed_gll

`struct minmea_sentence_gll parsed_nmea::parsed_gll`
Latitudine e Longitudine.
Definition at line 80 of file gps.h.

4.1.2.4 parsed_gsa

`struct minmea_sentence_gsa parsed_nmea::parsed_gsa`
DOP e satelliti attivi.
Definition at line 79 of file gps.h.

4.1.2.5 parsed_gst

`struct minmea_sentence_gst parsed_nmea::parsed_gst`
Statistiche rumore pseudorange.
Definition at line 81 of file gps.h.

4.1.2.6 parsed_gsv

`struct minmea_sentence_gsv parsed_nmea::parsed_gsv`
Satelliti in vista.
Definition at line 82 of file gps.h.

4.1.2.7 parsed_rmc

`struct minmea_sentence_rmc parsed_nmea::parsed_rmc`
Dati minimi raccomandati (Data, Ora, Posizione).
Definition at line 77 of file gps.h.

4.1.2.8 parsed_vtg

`struct minmea_sentence_vtg parsed_nmea::parsed_vtg`
Velocità e rotta reale.
Definition at line 83 of file gps.h.

4.1.2.9 parsed_zda

`struct minmea_sentence_zda parsed_nmea::parsed_zda`
Data e Ora UTC.
Definition at line 84 of file gps.h.
The documentation for this struct was generated from the following file:

- firm/include/gps.h

4.2 WeatherState Struct Reference

Contenitore globale per lo stato meteorologico.
`#include <weather.h>`

Data Fields

- bool curValid = false
- bool fcValid = false
- char city [24] = "NA"
- float curTempC = NAN
- float curWindKmh = NAN
- int curHumidity = -1
- int curWcode = -1
- uint8_t weekday0 = 0
- uint8_t recvMask = 0
- uint32_t lastUpdateMs = 0

Matrici Forecast (7 giorni x 24 ore)

- float tempC [7][24]
- uint8_t wcode [7][24]

4.2.1 Detailed Description

Contenitore globale per lo stato meteorologico.

- Memorizza i dati correnti (temperatura, vento, umidità) e una matrice 7x24 per le previsioni orarie della settimana. Utilizza una maschera di bit per tracciare la ricezione completa dei dati dal server.

Definition at line 22 of file weather.h.

4.2.2 Field Documentation

4.2.2.1 city

`char WeatherState::city[24] = "NA"`
Nome della città (max 23 caratteri + null terminator).
Definition at line 26 of file weather.h.

4.2.2.2 curHumidity

`int WeatherState::curHumidity = -1`
Percentuale di umidità attuale (0-100).
Definition at line 30 of file weather.h.

4.2.2.3 curTempC

`float WeatherState::curTempC = NAN`
Temperatura attuale in gradi Celsius.
Definition at line 28 of file weather.h.

4.2.2.4 curValid

`bool WeatherState::curValid = false`
True se i dati meteo correnti sono stati popolati correttamente.
Definition at line 23 of file weather.h.

4.2.2.5 curWcode

`int WeatherState::curWcode = -1`
Codice WMO delle condizioni meteo attuali.
Definition at line 31 of file weather.h.

4.2.2.6 curWindKmh

`float WeatherState::curWindKmh = NAN`
Velocità del vento attuale in km/h.
Definition at line 29 of file weather.h.

4.2.2.7 fcValid

`bool WeatherState::fcValid = false`
True se tutte le previsioni (7 giorni) sono state ricevute.
Definition at line 25 of file weather.h.

4.2.2.8 lastUpdateMs

`uint32_t WeatherState::lastUpdateMs = 0`
Timestamp (millis) dell'ultimo aggiornamento ricevuto.
Definition at line 43 of file weather.h.

4.2.2.9 recvMask

`uint8_t WeatherState::recvMask = 0`
Maschera bitwise: il bit i-esimo è 1 se il giorno i è stato ricevuto.
Definition at line 40 of file weather.h.

4.2.2.10 tempC

`float WeatherState::tempC[7][24]`
Matrice delle temperature orarie previste.
Definition at line 35 of file weather.h.

4.2.2.11 wcode

`uint8_t WeatherState::wcode[7][24]`
Matrice dei codici meteo orari previsti.
Definition at line 36 of file weather.h.

4.2.2.12 weekday0

`uint8_t WeatherState::weekday0 = 0`
Indice del giorno della settimana corrente (0=Lun, 6=Dom).
Definition at line 39 of file weather.h.
The documentation for this struct was generated from the following file:

- firm/include/weather.h

Chapter 5

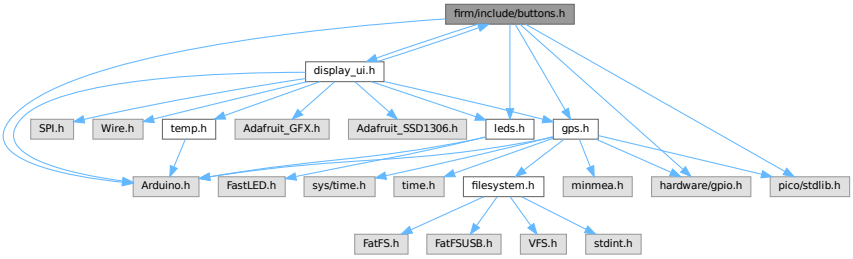
File Documentation

5.1 firm/include/buttons.h File Reference

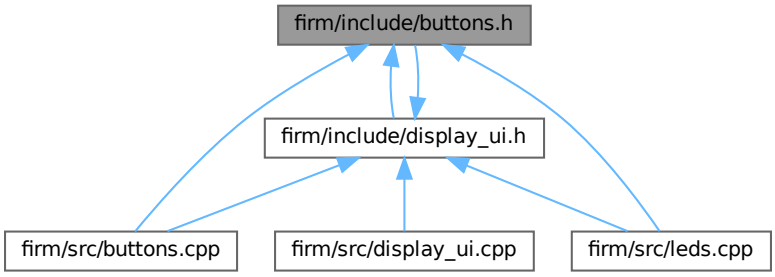
Definizioni e prototipi per la gestione degli input utente e della UI.

```
#include <Arduino.h>
#include "display_ui.h"
#include "gps.h"
#include "hardware/gpio.h"
#include "leds.h"
#include "pico/stdlib.h"
```

Include dependency graph for buttons.h:



This graph shows which files directly or indirectly include this file:



Macros

Mapping Hardware Pulsanti

- #define SW_UP 22
- #define SW_DOWN 23
- #define SW_RIGHT 24
- #define SW_LEFT 25

Enumerations

- enum KeyEvent : uint8_t {
  KEY_NONE , KEY_UP , KEY_DOWN , KEY_LEFT ,
  KEY_RIGHT }

  Tipi di eventi generabili dalla pressione dei pulsanti.

Functions

- void buttonsInit ()

  Inizializza i pin GPIO e configura gli interrupt per tutti i pulsanti.
- void buttonsUpdate ()

  Esegue il polling della logica dei menu e gestisce gli eventi dei tasti.
- void onLeftRightCombo ()

  Gestisce l'azione combinata (pressione simultanea) di LEFT e RIGHT. Tipicamente utilizzata per il toggle tra Home e Menu.
- bool checkLeftRightCombo (KeyEvent ev)

  Analizza l'evento corrente per verificare se fa parte di una combo.
- long long debounce_callback (alarm_id_t id, void ∗user_data)

  Funzione richiamata dall'allarme hardware per confermare la pressione (Debounce).
- void buttons_callback (uint gpio, uint32_t events)

  Callback centralizzato per gli interrupt GPIO dei pulsanti.

Variables

- int ledsNumber

  Identificatore per l'animazione dei LED di navigazione.

Variabili di Stato UI

- int menuIndex
- bool menuOpen
- bool inSubmenu
- bool ambientDisplay

Flag di Interrupt (Volatile)

- volatile bool up_press
- volatile bool down_press
- volatile bool left_press
- volatile bool right_press

Parametri Temporali e Telemetria

- uint32_t tLeft
- uint32_t tRight
- const uint32_t COMBO_MS
- uint32_t lastActivity
- const uint32_t DISPLAY_TIMEOUT_MS

5.1.1 Detailed Description

Definizioni e prototipi per la gestione degli input utente e della UI.

- Questo header contiene le definizioni dei pin per i pulsanti di navigazione, i flag di stato per la macchina a stati del menu e i prototipi per la gestione degli interrupt e del debounce su RP2040.

Definition in file buttons.h.

5.1.2 Macro Definition Documentation

5.1.2.1 SW_DOWN

```
#define SW_DOWN 23
```

Pin GPIO per il pulsante Giù.
Definition at line 23 of file buttons.h.

5.1.2.2 SW_LEFT

```
#define SW_LEFT 25
```

Pin GPIO per il pulsante Sinistra / Indietro.
Definition at line 25 of file buttons.h.

5.1.2.3 SW_RIGHT

```
#define SW_RIGHT 24
```

Pin GPIO per il pulsante Destra / Invio.
Definition at line 24 of file buttons.h.

5.1.2.4 SW_UP

```
#define SW_UP 22
```

Pin GPIO per il pulsante Su.
Definition at line 22 of file buttons.h.

5.1.3 Enumeration Type Documentation

5.1.3.1 KeyEvent

```
enum KeyEvent : uint8_t
```

Tipi di eventi generabili dalla pressione dei pulsanti.

Enumerator

KEY_NONE	Nessun tasto premuto.
KEY_UP	Evento tasto Su.
KEY_DOWN	Evento tasto Giù.
KEY_LEFT	Evento tasto Sinistra.
KEY_RIGHT	Evento tasto Destra.

Definition at line 65 of file buttons.h.

5.1.4 Function Documentation

5.1.4.1 buttons_callback()

```
void buttons_callback (
            uint gpio,
            uint32_t events )
```

Callback centralizzato per gli interrupt GPIO dei pulsanti.

Parameters

gpio	Numero del pin GPIO che ha scatenato l'IRQ.
events	Maschera degli eventi (rise/fall).

5.1.4.2 buttonsInit()

```
void buttonsInit ( )
```

Inizializza i pin GPIO e configura gli interrupt per tutti i pulsanti.
< Pin GPIO per il pulsante Su.
< Pin GPIO per il pulsante Giù.
< Pin GPIO per il pulsante Sinistra / Indietro.
< Pin GPIO per il pulsante Destra / Invio.
< Pin GPIO per il pulsante Su.
Definition at line 111 of file buttons.cpp.

5.1.4.3 buttonsUpdate()

```
void buttonsUpdate ( )
```

Esegue il polling della logica dei menu e gestisce gli eventi dei tasti.

Note

Deve essere chiamata ciclicamente nel loop principale.

Definition at line 131 of file buttons.cpp.

5.1.4.4 checkLeftRightCombo()

```
bool checkLeftRightCombo (
            KeyEvent ev )
```

Analizza l'evento corrente per verificare se fa parte di una combo.

Parameters

ev	L'evento tasto catturato.

Returns

true se l'evento è stato consumato come parte di una combo.

Definition at line 340 of file buttons.cpp.

5.1.4.5 debounce_callback()

```
long long debounce_callback (
            alarm_id_t id,
            void * user_data )
```

Funzione richiamata dall'allarme hardware per confermare la pressione (Debounce).

Parameters

id	ID dell'allarme hardware Pico.
user_data	Dati utente (generalmente il numero del pin).

Returns

long long Tempo per la prossima esecuzione (0 per esecuzione singola).

< Pin GPIO per il pulsante Su.
< Pin GPIO per il pulsante Giù.
< Pin GPIO per il pulsante Sinistra / Indietro.
< Pin GPIO per il pulsante Destra / Invio.
Definition at line 69 of file buttons.cpp.

5.1.4.6 onLeftRightCombo()

`void onLeftRightCombo ( )`
Gestisce l'azione combinata (pressione simultanea) di LEFT e RIGHT. Tipicamente utilizzata per il toggle tra Home e Menu.
Definition at line 367 of file buttons.cpp.

5.1.5 Variable Documentation

5.1.5.1 ambientDisplay

`bool ambientDisplay [extern]`
Indica se il sistema è in modalità Home (Ambient).
Stato del display: true se in modalità risparmio/home.
Definition at line 11 of file buttons.cpp.

5.1.5.2 COMBO_MS

`const uint32_t COMBO_MS [extern]`
Finestra temporale massima per validare una pressione simultanea.
Finestra temporale (ms) per rilevare una pressione simultanea.
Definition at line 20 of file buttons.cpp.

5.1.5.3 DISPLAY_TIMEOUT_MS

`const uint32_t DISPLAY_TIMEOUT_MS [extern]`
Tempo di inattività prima del ritorno alla home.
Timeout (ms) per il ritorno automatico alla Home.
Definition at line 23 of file buttons.cpp.

5.1.5.4 down_press

`volatile bool down_press [extern]`
Segnala l'avvenuto debounce del tasto DOWN.
Flag IRQ per pulsante DOWN.
Definition at line 14 of file buttons.cpp.

5.1.5.5 inSubmenu

`bool inSubmenu [extern]`
Indica se l'utente si trova all'interno di una schermata sottomenu.
Flag di presenza all'interno di un sottomenu.
Definition at line 10 of file buttons.cpp.

5.1.5.6 lastActivity

`uint32_t lastActivity [extern]`
Timestamp dell'ultima attività registrata per il timeout.
Timestamp dell'ultima interazione utente.
Definition at line 22 of file buttons.cpp.

5.1.5.7 ledsNumber

`int ledsNumber [extern]`
Identificatore per l'animazione dei LED di navigazione.
Identificativo LED per animazione navigazione.
Definition at line 28 of file buttons.cpp.

5.1.5.8 left_press

`volatile bool left_press [extern]`
Segnala l'avvenuto debounce del tasto LEFT.
Flag IRQ per pulsante LEFT.
Definition at line 15 of file buttons.cpp.

5.1.5.9 menuIndex

`int menuIndex [extern]`
Indice corrente della voce selezionata nel menu principale.
Indice della voce di menu selezionata.
Definition at line 8 of file buttons.cpp.

5.1.5.10 menuOpen

`bool menuOpen [extern]`
Stato di visibilità del menu principale.
Stato di apertura del menu principale.
Definition at line 9 of file buttons.cpp.

5.1.5.11 right_press

`volatile bool right_press [extern]`
Segnala l'avvenuto debounce del tasto RIGHT.
Flag IRQ per pulsante RIGHT.
Definition at line 16 of file buttons.cpp.

5.1.5.12 tLeft

`uint32_t tLeft [extern]`
Timestamp dell'ultima pressione sinistra per rilevamento combo.
Definition at line 18 of file buttons.cpp.

5.1.5.13 tRight

`uint32_t tRight [extern]`
Timestamp dell'ultima pressione destra per rilevamento combo.
Definition at line 19 of file buttons.cpp.

5.1.5.14 up_press

`volatile bool up_press [extern]`
Segnala l'avvenuto debounce del tasto UP.
Flag IRQ per pulsante UP.
Definition at line 13 of file buttons.cpp.

5.2 buttons.h

Go to the documentation of this file.

```
00001
00009 #ifndef BUTTONS_H
00010 #define BUTTONS_H
00011
00012 #include <Arduino.h>
00013
00014 #include "display_ui.h"
00015 #include "gps.h"
00016 #include "hardware/gpio.h"
00017 #include "leds.h"
00018 #include "pico/stdlib.h"
00019
00022 #define SW_UP    22
```

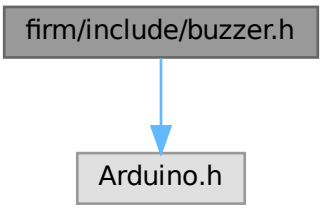
```
#define SW_DOWN  23
#define SW_RIGHT 24
#define SW_LEFT  25
extern int menuIndex;
extern bool menuOpen;
extern bool inSubmenu;
extern bool ambientDisplay;
extern volatile bool up_press;
extern volatile bool down_press;
extern volatile bool left_press;
extern volatile bool right_press;
extern uint32_t tLeft;
extern uint32_t tRight;
extern const uint32_t COMBO_MS;
extern uint32_t lastActivity;
extern const uint32_t DISPLAY_TIMEOUT_MS;
extern int ledsNumber;

enum KeyEvent : uint8_t {
    KEY_NONE,
    KEY_UP,
    KEY_DOWN,
    KEY_LEFT,
    KEY_RIGHT
};

/* --- Funzioni Core --- */

void buttonsInit();

void buttonsUpdate();

void onLeftRightCombo();

bool checkLeftRightCombo(KeyEvent ev);

/* --- Prototipi Callback --- */

long long debounce_callback(alarm_id_t id, void* user_data);

void buttons_callback(uint gpio, uint32_t events);

#endif  // BUTTONS_H
```

5.3 firm/include/buzzer.h File Reference

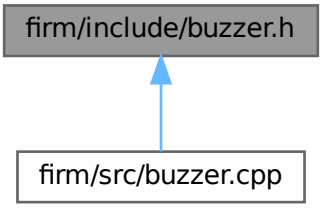
Definizioni e prototipi per il controllo del feedback sonoro.

`#include <Arduino.h>`

Include dependency graph for buzzer.h:



This graph shows which files directly or indirectly include this file:



Macros

- #define BUZZER 21

  Pin GPIO predefinito per il collegamento del buzzer.

Functions

- void buzzerInit (uint8_t pin)

  Configura il pin del buzzer e prepara l'hardware.
- void buzzerBeep (uint16_t freq=1200, uint16_t duration=60)

  Genera un suono impulsivo (beep).

5.3.1 Detailed Description

Definizioni e prototipi per il controllo del feedback sonoro.

- Questo modulo permette di gestire un buzzer piezoelettrico per fornire feedback acustico all'utente durante la navigazione dei menu o eventi di sistema.

Definition in file buzzer.h.

5.3.2 Macro Definition Documentation

5.3.2.1 BUZZER

```
#define BUZZER 21
```

Pin GPIO predefinito per il collegamento del buzzer.

-

Definition at line 16 of file buzzer.h.

5.3.3 Function Documentation

5.3.3.1 buzzerBeep()

```
void buzzerBeep (
            uint16_t freq = 1200,
            uint16_t duration = 60 )
```

Genera un suono impulsivo (beep).

- Produce un tono alla frequenza desiderata per una durata specifica. La funzione è bloccante per l'intera durata del suono.

Parameters

freq	Frequenza del tono in Hertz. Valore predefinito: `1200` Hz.
duration	Durata del suono in millisecondi. Valore predefinito: `60` ms.

-

Definition at line 22 of file buzzer.cpp.

5.3.3.2 buzzerInit()

```
void buzzerInit (
            uint8_t pin )
```

Configura il pin del buzzer e prepara l'hardware.

- Imposta il pin specificato come output. Se configurato nelle macro globali, può emettere un suono di conferma all'avvio.

Parameters

pin	Il numero del pin GPIO da inizializzare.

-

Definition at line 5 of file buzzer.cpp.

5.4 buzzer.h

Go to the documentation of this file.

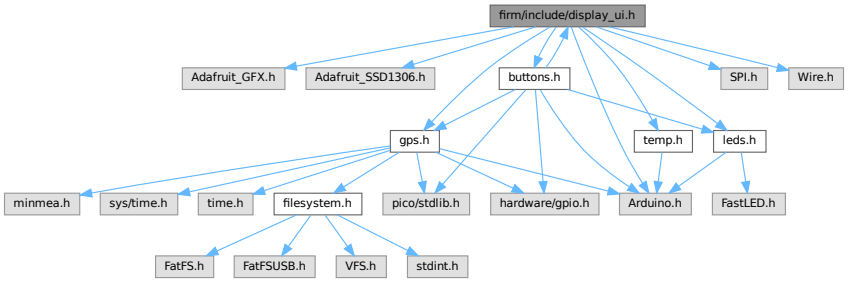
```
00001
00009 #ifndef BUZZER_H
00010 #define BUZZER_H
00011
00012 #include <Arduino.h>
00013
00016 #define BUZZER 21
00017
00024 void buzzerInit(uint8_t pin);
00025
00034 void buzzerBeep(uint16_t freq = 1200, uint16_t duration = 60);
00035
00036 #endif  // BUZZER_H
```

5.5 firm/include/display_ui.h File Reference

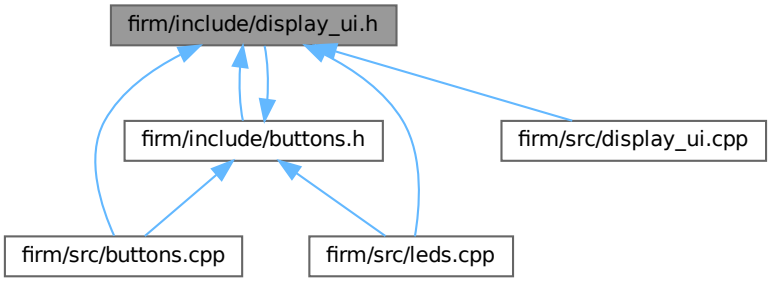
Definizioni hardware e prototipi per l'interfaccia utente OLED.

```
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <Arduino.h>
#include <SPI.h>
#include <Wire.h>
#include "buttons.h"
#include "gps.h"
#include "leds.h"
#include "temp.h"
```

Include dependency graph for display_ui.h:



This graph shows which files directly or indirectly include this file:



Macros

Configurazione Hardware Display

- #define DISPLAY_SDA_I2C_PIN 0
- #define DISPLAY_SCL_I2C_PIN 1
- #define SCREEN_WIDTH 128
- #define SCREEN_HEIGHT 64
- #define OLED_RESET -1
- #define SCREEN_ADDRESS 0x3C

Functions

- bool displayInit ()

  Inizializza il bus I2C e il controller SSD1306.
- void updateDisplayTimeout ()

  Gestisce il timeout dell'inattività e il ritorno automatico alla Home.
- void setBrightness (uint8_t level)

  Regola l'intensità luminosa (contrasto) del pannello OLED.
- void drawHomeScreen ()

  Renderizza la schermata Home con ora, data e meteo corrente.
- void drawMenu (int index)

  Disegna il menu principale con lo stile selezionato in ScreenStyle.
- void drawMenu_evi (int index)

Renderizza forzatamente il menu con stile evidenziato (testo invertito).

- void drawSelected (const char *title)

  Mostra una schermata di segnaposto per le voci selezionate.
- void drawSettingsScreen ()

  Renderizza la schermata di configurazione impostazioni.
- void drawMeteoScreen ()

  Renderizza la schermata dettagliata delle previsioni meteo.
- void drawSystemScreen ()

  Renderizza la schermata di stato del sistema.
- void drawInfoScreen ()

  Renderizza la schermata crediti e informazioni.
- void drawGPSScreen ()

  Renderizza la schermata dei dati GPS in tempo reale.

Variables

Configurazione Estetica e Navigazione

- uint8_t ScreenStyle
- const char * menuItems [ ]
- const int menuLength
- struct parsed_nmea gpsData

Indici di Navigazione Meteo

- uint8_t gpsDayIndex
- uint8_t gpsHourIndex

5.5.1 Detailed Description

Definizioni hardware e prototipi per l'interfaccia utente OLED.

- Questo header definisce la configurazione del display SSD1306 (pin, dimensioni, indirizzi) e fornisce l'interfaccia per il rendering delle diverse schermate applicative.

Definition in file display_ui.h.

5.5.2 Macro Definition Documentation

5.5.2.1 DISPLAY_SCL_I2C_PIN

```
#define DISPLAY_SCL_I2C_PIN 1
```

Pin GPIO per segnale I2C SCL.
Definition at line 26 of file display_ui.h.

5.5.2.2 DISPLAY_SDA_I2C_PIN

```
#define DISPLAY_SDA_I2C_PIN 0
```

Pin GPIO per segnale I2C SDA.
Definition at line 25 of file display_ui.h.

5.5.2.3 OLED_RESET

```
#define OLED_RESET -1
```

Pin di reset (non utilizzato/non disponibile).
Definition at line 29 of file display_ui.h.

5.5.2.4 SCREEN_ADDRESS

```
#define SCREEN_ADDRESS  0x3C
```

Indirizzo I2C standard per display SSD1306. \
Definition at line 32 of file display_ui.h.

5.5.2.5 SCREEN_HEIGHT

```
#define SCREEN_HEIGHT 64
```

Altezza del display OLED in pixel.
Definition at line 28 of file display_ui.h.

5.5.2.6 SCREEN_WIDTH

```
#define SCREEN_WIDTH 128
```

Larghezza del display OLED in pixel.
Definition at line 27 of file display_ui.h.

5.5.3 Function Documentation

5.5.3.1 displayInit()

```
bool displayInit ( )
```

Inizializza il bus I2C e il controller SSD1306.

Returns

true Se il display è stato allocato correttamente, false in caso di errore.

< Pin GPIO per segnale I2C SDA.
< Pin GPIO per segnale I2C SCL.
< Indirizzo I2C standard per display SSD1306.

Definition at line 23 of file display_ui.cpp.

5.5.3.2 drawGPSScreen()

```
void drawGPSScreen ( )
```

Renderizza la schermata dei dati GPS in tempo reale.
Definition at line 245 of file display_ui.cpp.

5.5.3.3 drawHomeScreen()

```
void drawHomeScreen ( )
```

Renderizza la schermata Home con ora, data e meteo corrente.
Definition at line 50 of file display_ui.cpp.

5.5.3.4 drawInfoScreen()

```
void drawInfoScreen ( )
```

Renderizza la schermata crediti e informazioni.
Definition at line 322 of file display_ui.cpp.

5.5.3.5 drawMenu()

```
void drawMenu (
            int index )
```

Disegna il menu principale con lo stile selezionato in ScreenStyle.

Parameters

index	Indice della voce di menu da evidenziare.

Definition at line 97 of file display_ui.cpp.

5.5.3.6 drawMenu_evi()

```
void drawMenu_evi (
            int index )
```

Renderizza forzatamente il menu con stile evidenziato (testo invertito).

Parameters

index	Indice della voce di menu da evidenziare.

Definition at line 118 of file display_ui.cpp.

5.5.3.7 drawMeteoScreen()

```
void drawMeteoScreen ( )
```

Renderizza la schermata dettagliata delle previsioni meteo.
Definition at line 170 of file display_ui.cpp.

5.5.3.8 drawSelected()

```
void drawSelected (
            const char * title )
```

Mostra una schermata di segnaposto per le voci selezionate.

Parameters

title	Titolo o nome della sezione da visualizzare.

Definition at line 308 of file display_ui.cpp.

5.5.3.9 drawSettingsScreen()

```
void drawSettingsScreen ( )
```

Renderizza la schermata di configurazione impostazioni.
Definition at line 151 of file display_ui.cpp.

5.5.3.10 drawSystemScreen()

```
void drawSystemScreen ( )
```

Renderizza la schermata di stato del sistema.
Definition at line 319 of file display_ui.cpp.

5.5.3.11 setBrightness()

```
void setBrightness (
            uint8_t level )
```

Regola l'intensità luminosa (contrasto) del pannello OLED.

Parameters

level	Valore di contrasto da 0 (min) a 255 (max).

Definition at line 337 of file display_ui.cpp.

5.5.3.12 updateDisplayTimeout()

```
void updateDisplayTimeout ( )
```

Gestisce il timeout dell'inattività e il ritorno automatico alla Home.
Definition at line 326 of file display_ui.cpp.

5.5.4 Variable Documentation

5.5.4.1 gpsData

```
struct parsed_nmea gpsData [extern]
```
Struttura globale contenente i dati telemetrici GPS correnti.

5.5.4.2 gpsDayIndex

```
uint8_t gpsDayIndex [extern]
```
Offset del giorno selezionato per il forecast (0-6).
Indice del giorno selezionato per le previsioni meteo.
Definition at line 12 of file display_ui.cpp.

5.5.4.3 gpsHourIndex

```
uint8_t gpsHourIndex [extern]
```
Offset dell'ora selezionata per il forecast (0-23).
Indice dell'ora selezionata per le previsioni meteo.
Definition at line 13 of file display_ui.cpp.

5.5.4.4 menuItems

```
const char* menuItems[] [extern]
```
Array di stringhe contenente i nomi delle voci del menu.
Definition at line 17 of file display_ui.cpp.

5.5.4.5 menuLength

```
const int menuLength [extern]
```
Numero totale di elementi nel menu.
Definition at line 19 of file display_ui.cpp.

5.5.4.6 ScreenStyle

```
uint8_t ScreenStyle [extern]
```
Stile visivo: 1 = Semplice (testo), 0 = Highlight (barra invertita).
Stile del menu: 1 = Semplice, 0 = Evidenziato (Highlight).
Definition at line 15 of file display_ui.cpp.

5.6 display_ui.h

Go to the documentation of this file.

```
00001
00009 #ifndef DISPLAY_UI_H
00010 #define DISPLAY_UI_H
00011
00012 #include <Adafruit_GFX.h>
00013 #include <Adafruit_SSD1306.h>
00014 #include <Arduino.h>
00015 #include <SPI.h>
00016 #include <Wire.h>
00017
00018 #include "buttons.h"
00019 #include "gps.h"
00020 #include "leds.h"
00021 #include "temp.h"
00022
00025 #define DISPLAY_SDA_I2C_PIN 0
00026 #define DISPLAY_SCL_I2C_PIN 1
00027 #define SCREEN_WIDTH        128
00028 #define SCREEN_HEIGHT       64
```

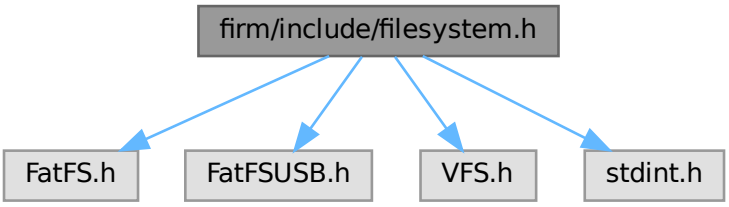
```
#define OLED_RESET          -1
#define SCREEN_ADDRESS                                          \
    0x3C
extern uint8_t ScreenStyle;
extern const char* menuItems[];
extern const int menuLength;
extern struct parsed_nmea gpsData;
extern uint8_t gpsDayIndex;
extern uint8_t gpsHourIndex;
/* --- Funzioni di Ciclo Vita --- */

bool displayInit();

void updateDisplayTimeout();

void setBrightness(uint8_t level);

/* --- Funzioni di Rendering (Schermate) --- */

void drawHomeScreen();

void drawMenu(int index);

void drawMenu_evi(int index);

void drawSelected(const char* title);

void drawSettingsScreen();

void drawMeteoScreen();

void drawSystemScreen();

void drawInfoScreen();

void drawGPSScreen();

#endif  // DISPLAY_UI_H
```

5.7 firm/include/filesystem.h File Reference

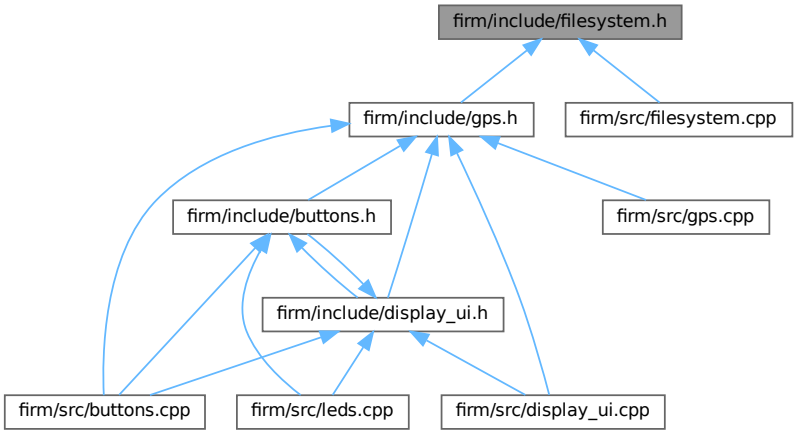
Interfaccia per la gestione del filesystem FatFS e l'integrazione USB Mass Storage.

```
#include <FatFS.h>
#include <FatFSUSB.h>
#include <VFS.h>
#include <stdint.h>
```

Include dependency graph for filesystem.h:



This graph shows which files directly or indirectly include this file:



Functions

- bool FatFS_Setup ()

  Inizializza il filesystem FAT sulla Flash interna.
- bool FatFSUSB_Setup ()

  Configura e avvia il servizio USB Mass Storage.
- void unplug (uint32_t i)

  Callback richiamato al distacco del cavo USB.
- void plug (uint32_t i)

  Callback richiamato quando il PC monta l'unità.
- bool mountable (uint32_t i)

  Funzione di verifica per il mount sicuro su PC.
- bool sync_files ()

  Gestisce la logica di sincronizzazione post-connessione.

Variables

- volatile bool updated

  Flag di notifica per il ricaricamento dei file.
- volatile bool driveConnected

  Stato della connessione USB.
- volatile bool inPrinting

  Semaforo di accesso al filesystem.

5.7.1 Detailed Description

Interfaccia per la gestione del filesystem FatFS e l'integrazione USB Mass Storage.

- Questo header definisce le funzioni e i flag globali necessari per gestire la memoria flash interna come disco FAT, permettendo la condivisione sicura dei dati tra il firmware dell'RP2040 e un PC esterno.

Definition in file filesystem.h.

5.7.2 Function Documentation

5.7.2.1 FatFS_Setup()

```
bool FatFS_Setup ( )
```

Inizializza il filesystem FAT sulla Flash interna.

Returns

- true se il mount (o l'eventuale formattazione) è riuscito.

Definition at line 9 of file filesystem.cpp.

5.7.2.2 FatFSUSB_Setup()

```
bool FatFSUSB_Setup ( )
```

Configura e avvia il servizio USB Mass Storage.

Returns

- true se il dispositivo USB è stato inizializzato correttamente.

Definition at line 25 of file filesystem.cpp.

5.7.2.3 mountable()

```
bool mountable (
            uint32_t i )
```

Funzione di verifica per il mount sicuro su PC.

- Controlla il flag `inPrinting` per decidere se concedere l'accesso al PC.

Parameters

i	Identificativo dell'interfaccia USB.

Returns

- true se il filesystem è libero e montabile dall'host.

Definition at line 54 of file filesystem.cpp.

5.7.2.4 plug()

```
void plug (
            uint32_t i )
```

Callback richiamato quando il PC monta l'unità.

Parameters

i	Identificativo dell'interfaccia USB.

Definition at line 48 of file filesystem.cpp.

5.7.2.5 sync_files()

```
bool sync_files ( )
```

Gestisce la logica di sincronizzazione post-connessione.

- Se il disco è stato modificato dal PC, questa funzione si occupa di ripristinare l'accesso locale.

Returns

true se il filesystem è pronto per l'uso da parte del firmware.

Definition at line 59 of file filesystem.cpp.

5.7.2.6 unplug()

```
void unplug (
            uint32_t i )
```

Callback richiamato al distacco del cavo USB.

Parameters

i	Identificativo dell'interfaccia USB (fornito dal framework).

Definition at line 42 of file filesystem.cpp.

5.7.3 Variable Documentation

5.7.3.1 driveConnected

```
volatile bool driveConnected [extern]
```

Stato della connessione USB.

- Vale `true` se il PC ha preso il controllo del filesystem (mount effettuato).

Definition at line 5 of file filesystem.cpp.

5.7.3.2 inPrinting

```
volatile bool inPrinting [extern]
```

Semaforo di accesso al filesystem.

- Vale `true` quando il firmware sta scrivendo o leggendo attivamente dalla flash. Impedisce al PC di montare il disco contemporaneamente per evitare corruzione.

Definition at line 7 of file filesystem.cpp.

5.7.3.3 updated

```
volatile bool updated [extern]
```

Flag di notifica per il ricaricamento dei file.

- Viene impostato a `true` quando il disco viene scollegato dal PC, segnalando al firmware di rieseguire il mount per leggere i nuovi dati.

Definition at line 3 of file filesystem.cpp.

5.8 filesystem.h

Go to the documentation of this file.

```
00001
00010 #ifndef FILESYSTEM_H
00011 #define FILESYSTEM_H
00012
00013 #include <FatFS.h>
00014 #include <FatFSUSB.h>
00015 #include <VFS.h>
00016 #include <stdint.h>
00017
00018 /* --- Variabili di stato globali --- */
00019
00025 extern volatile bool updated;
00026
```

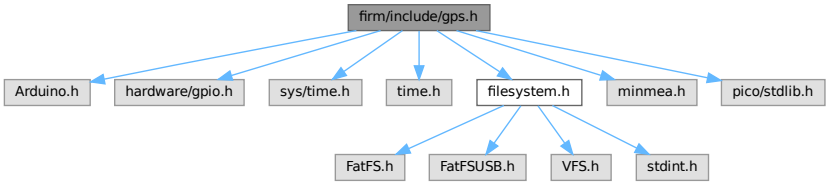
```
extern volatile bool driveConnected;

extern volatile bool inPrinting;

/* --- Funzioni di Configurazione --- */

bool FatFS_Setup();

bool FatFSUSB_Setup();

/* --- Callback e Gestione Risorse --- */

void unplug(uint32_t i);

void plug(uint32_t i);

bool mountable(uint32_t i);

bool sync_files();

#endif  // FILESYSTEM_H
```

5.9 firm/include/gps.h File Reference

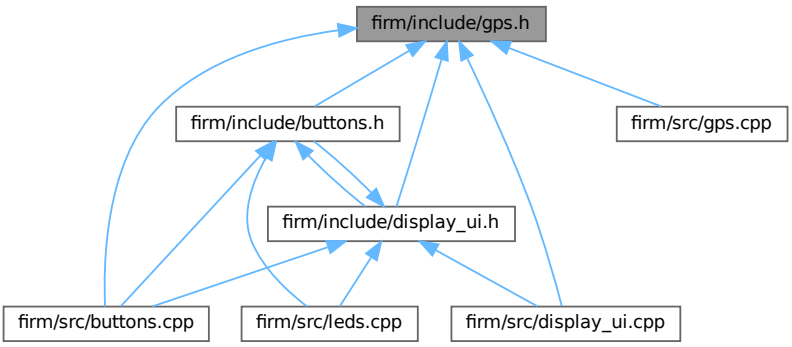
Definizioni, macro e strutture per la gestione della telemetria GPS.

```
#include <Arduino.h>
#include <hardware/gpio.h>
#include <sys/time.h>
#include <time.h>
#include "filesystem.h"
#include "minmea.h"
#include "pico/stdlib.h"
```

Include dependency graph for gps.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct parsed_nmea

  Container globale per le sentenze NMEA parsate.

Macros

Mappatura Pin UART0

Necessaria per evitare sovrapposizioni con UART1 su GPIO 4 e 5.

- #define UART0_TX_PIN 16
- #define UART0_RX_PIN 17

Parametri Protocollo NMEA

- #define MINMEA_MAX_SENTENCE_LENGTH 82
- #define GPS_ACQUIRE_MAX_TRIES 30

Pinout Hardware GPS

- #define GPS_PPS_PIN 19
- #define GPS_EN_PIN 18
- #define UART1_TX_PIN 4
- #define UART1_RX_PIN 5
- #define GPSBAUD 9600
- #define LED_ALIVE 20

Timing e Filesystem

- #define GPS_SYNC_TIMEOUT_MSEC 20
- #define GPS_LOG_PATH "/gps_log.csv"
- #define GPS_LAST_PATH "/gps_last.csv"
- #define GPS_MAX_LINES 20
- #define GPS_WRITE_FREQ 600000

Protocollo Inter-Core

- #define GPS_SYNC_REQ 0xA
- #define GPS_SYNC_ACK 0xB

Functions

- bool gpsInit ()

  Configura GPIO e UART per il modulo GPS.
- bool nmea_gps_parse (String *nmea_message, struct parsed_nmea *nmea_ptr)

  Esegue il parsing di una stringa NMEA.
- bool gpsAcquire (enum minmea_sentence_id sentence_type, struct parsed_nmea *nmea_ptr)

  Cerca e acquisisce una specifica sentenza dalla seriale.
- bool GetDate_and_Time (struct parsed_nmea *nmea_ptr)

  Estrae data e ora dai messaggi RMC.
- bool GetPosition_and_Satellites (struct parsed_nmea *nmea_ptr)

  Estrae posizione e numero satelliti dai messaggi GGA.
- bool GPS_sync (struct parsed_nmea *nmea_ptr, bool force)

  Sincronizza lo stato globale e gestisce il salvataggio su disco.
- bool save_gps_last (struct parsed_nmea *nmea_ptr)

  Sovrascrive il file dell'ultimo fix noto.
- bool save_gps_log20 (struct parsed_nmea *nmea_ptr)

  Aggiunge una riga al log rotativo (max 20 righe).
- void check_GPS_sync_req (queue_t *recv_queue, queue_t *send_queue, struct parsed_nmea *nmea_ptr)

Gestisce le richieste di sync provenienti dall'altro core.

- void print_NMEA_rmc (void ∗frame)
- void print_NMEA_gga (void ∗frame)
- void print_NMEA_gst (void ∗frame)
- void print_NMEA_gsv (void ∗frame)
- void print_NMEA_vtg (void ∗frame)
- void print_NMEA_zda (void ∗frame)

Variables

- uint32_t lastUiGpsMs
- volatile bool gpsDirty

5.9.1 Detailed Description

Definizioni, macro e strutture per la gestione della telemetria GPS.

- Questo header definisce l'interfaccia per il modulo GPS, incluse le mappature dei pin UART per RP2040, le impostazioni di logging su filesystem e la struttura dati che aggrega i risultati del parsing della libreria minmea.

Definition in file gps.h.

5.9.2 Macro Definition Documentation

5.9.2.1 GPS_ACQUIRE_MAX_TRIES

```
#define GPS_ACQUIRE_MAX_TRIES  30
```

Tentativi massimi di lettura sentenze per ogni chiamata gpsAcquire. \
Definition at line 29 of file gps.h.

5.9.2.2 GPS_EN_PIN

```
#define GPS_EN_PIN 18
```

Pin di abilitazione alimentazione modulo (Attivo Basso).
Definition at line 35 of file gps.h.

5.9.2.3 GPS_LAST_PATH

```
#define GPS_LAST_PATH "/gps_last.csv"
```

Percorso del file con l'ultima posizione nota.
Definition at line 46 of file gps.h.

5.9.2.4 GPS_LOG_PATH

```
#define GPS_LOG_PATH "/gps_log.csv"
```

Percorso del file log rotativo (CSV).
Definition at line 45 of file gps.h.

5.9.2.5 GPS_MAX_LINES

```
#define GPS_MAX_LINES 20
```

Numero massimo di righe per il log rotativo.
Definition at line 47 of file gps.h.

5.9.2.6 GPS_PPS_PIN

```
#define GPS_PPS_PIN 19
```

Pin per il segnale Pulse Per Second (PPS) dal GPS.
Definition at line 34 of file gps.h.

5.9.2.7 GPS_SYNC_ACK

```
#define GPS_SYNC_ACK 0xB
```

Risposta di avvenuta sincronizzazione.
Definition at line 56 of file gps.h.

5.9.2.8 GPS_SYNC_REQ

```
#define GPS_SYNC_REQ 0xA
```

Comando di richiesta sincronizzazione GPS.
Definition at line 55 of file gps.h.

5.9.2.9 GPS_SYNC_TIMEOUT_MSEC

```
#define GPS_SYNC_TIMEOUT_MSEC 20
```

Timeout per la sincronizzazione di data e ora.
Definition at line 44 of file gps.h.

5.9.2.10 GPS_WRITE_FREQ

```
#define GPS_WRITE_FREQ  600000
```

Frequenza minima di scrittura su Flash (ms) per protezione \ usura.
Definition at line 50 of file gps.h.

5.9.2.11 GPSBAUD

```
#define GPSBAUD 9600
```

Baud rate predefinito del modulo GPS.
Definition at line 38 of file gps.h.

5.9.2.12 LED_ALIVE

```
#define LED_ALIVE 20
```

GPIO per il LED di segnalazione attività (Heartbeat).
Definition at line 39 of file gps.h.

5.9.2.13 MINMEA_MAX_SENTENCE_LENGTH

```
#define MINMEA_MAX_SENTENCE_LENGTH 82
```

Lunghezza massima di una riga NMEA standard.
Definition at line 26 of file gps.h.

5.9.2.14 UART0_RX_PIN

```
#define UART0_RX_PIN 17
```

Definition at line 21 of file gps.h.

5.9.2.15 UART0_TX_PIN

```
#define UART0_TX_PIN 16
```

Definition at line 20 of file gps.h.

5.9.2.16 UART1_RX_PIN

```
#define UART1_RX_PIN 5
```

Pin RX per comunicazione UART con il modulo.
Definition at line 37 of file gps.h.

5.9.2.17 UART1_TX_PIN

```
#define UART1_TX_PIN 4
```

Pin TX per comunicazione UART con il modulo.
Definition at line 36 of file gps.h.

5.9.3 Function Documentation

5.9.3.1 check_GPS_sync_req()

```
void check_GPS_sync_req (
            queue_t * recv_queue,
            queue_t * send_queue,
            struct parsed_nmea * nmea_ptr )
```

Gestisce le richieste di sync provenienti dall'altro core.
Definition at line 395 of file gps.cpp.

5.9.3.2 GetDate_and_Time()

```
bool GetDate_and_Time (
            struct parsed_nmea * nmea_ptr )
```

Estrae data e ora dai messaggi RMC.
Definition at line 158 of file gps.cpp.

5.9.3.3 GetPosition_and_Satellites()

```
bool GetPosition_and_Satellites (
            struct parsed_nmea * nmea_ptr )
```

Estrae posizione e numero satelliti dai messaggi GGA.
Definition at line 148 of file gps.cpp.

5.9.3.4 GPS_sync()

```
bool GPS_sync (
            struct parsed_nmea * nmea_ptr,
            bool force )
```

Sincronizza lo stato globale e gestisce il salvataggio su disco.
< Timeout per la sincronizzazione di data e ora.
< Frequenza minima di scrittura su Flash (ms) per protezione usura.
Definition at line 168 of file gps.cpp.

5.9.3.5 gpsAcquire()

```
bool gpsAcquire (
            enum minmea_sentence_id sentence_type,
            struct parsed_nmea * nmea_ptr )
```

Cerca e acquisisce una specifica sentenza dalla seriale.
< GPIO per il LED di segnalazione attività (Heartbeat).
< Tentativi massimi di lettura sentenze per ogni chiamata gpsAcquire.

< GPIO per il LED di segnalazione attività (Heartbeat).
Definition at line 123 of file gps.cpp.

5.9.3.6 gpsInit()

```
bool gpsInit ( )
```

Configura GPIO e UART per il modulo GPS.
< GPIO per il LED di segnalazione attività (Heartbeat).
< GPIO per il LED di segnalazione attività (Heartbeat).
< GPIO per il LED di segnalazione attività (Heartbeat).

< Pin di abilitazione alimentazione modulo (Attivo Basso).
< Pin di abilitazione alimentazione modulo (Attivo Basso).
< Pin di abilitazione alimentazione modulo (Attivo Basso).
< Pin per il segnale Pulse Per Second (PPS) dal GPS.
< Pin per il segnale Pulse Per Second (PPS) dal GPS.
< Pin TX per comunicazione UART con il modulo.
< Pin RX per comunicazione UART con il modulo.
Definition at line 3 of file gps.cpp.

5.9.3.7 nmea_gps_parse()

```
bool nmea_gps_parse (
            String * nmea_message,
            struct parsed_nmea * nmea_ptr )
```

Esegue il parsing di una stringa NMEA.
Definition at line 53 of file gps.cpp.

5.9.3.8 print_NMEA_gga()

```
void print_NMEA_gga (
            void * frame )
```

Definition at line 307 of file gps.cpp.

5.9.3.9 print_NMEA_gst()

```
void print_NMEA_gst (
            void * frame )
```

Definition at line 313 of file gps.cpp.

5.9.3.10 print_NMEA_gsv()

```
void print_NMEA_gsv (
            void * frame )
```

Definition at line 338 of file gps.cpp.

5.9.3.11 print_NMEA_rmc()

```
void print_NMEA_rmc (
            void * frame )
```

Definition at line 289 of file gps.cpp.

5.9.3.12 print_NMEA_vtg()

```
void print_NMEA_vtg (
            void * frame )
```

Definition at line 362 of file gps.cpp.

5.9.3.13 print_NMEA_zda()

```
void print_NMEA_zda (
            void * frame )
```

Definition at line 352 of file gps.cpp.

5.9.3.14 save_gps_last()

```
bool save_gps_last (
            struct parsed_nmea * nmea_ptr )
```

Sovrascrive il file dell'ultimo fix noto.
< Percorso del file con l'ultima posizione nota.
Definition at line 194 of file gps.cpp.

5.9.3.15 save_gps_log20()

```
bool save_gps_log20 (
            struct parsed_nmea * nmea_ptr )
```

Aggiunge una riga al log rotativo (max 20 righe).
< Numero massimo di righe per il log rotativo.
< Percorso del file log rotativo (CSV).
< Numero massimo di righe per il log rotativo.
< Numero massimo di righe per il log rotativo.
< Numero massimo di righe per il log rotativo.
< Numero massimo di righe per il log rotativo.
< Numero massimo di righe per il log rotativo.
< Numero massimo di righe per il log rotativo.
< Percorso del file log rotativo (CSV).
Definition at line 218 of file gps.cpp.

5.9.4 Variable Documentation

5.9.4.1 gpsDirty

```
volatile bool gpsDirty  [extern]
```

Flag per indicare dati GPS non ancora processati.

5.9.4.2 lastUiGpsMs

```
uint32_t lastUiGpsMs  [extern]
```

Timestamp dell'ultimo aggiornamento UI dei dati GPS.

5.10 gps.h

Go to the documentation of this file.

```
00001
00009 #ifndef GPS_H
00010 #define GPS_H
00011
00012 #ifndef DEBUG
00014 #define DEBUG 1
00015 #endif
00016
00020 #define UART0_TX_PIN 16
00021 #define UART0_RX_PIN 17
00026 #define MINMEA_MAX_SENTENCE_LENGTH 82
00027 #define GPS_ACQUIRE_MAX_TRIES                                                    \
00028     30
00034 #define GPS_PPS_PIN  19
00035 #define GPS_EN_PIN   18
00036 #define UART1_TX_PIN 4
00037 #define UART1_RX_PIN 5
00038 #define GPSBAUD      9600
00039 #define LED_ALIVE    20
00044 #define GPS_SYNC_TIMEOUT_MSEC 20
00045 #define GPS_LOG_PATH          "/gps_log.csv"
00046 #define GPS_LAST_PATH         "/gps_last.csv"
00047 #define GPS_MAX_LINES         20
00048 #define GPS_WRITE_FREQ                                                           \
00049     600000
00055 #define GPS_SYNC_REQ 0xA
00056 #define GPS_SYNC_ACK 0xB
00059 #include <Arduino.h>
00060 #include <hardware/gpio.h>
00061 #include <sys/time.h>
00062 #include <time.h>
00063
00064 #include "filesystem.h"
00065 #include "minmea.h"
00066 #include "pico/stdlib.h"
00067
00075 struct parsed_nmea {
00076     struct minmea_sentence_gbs parsed_gbs;
00077     struct minmea_sentence_rmc parsed_rmc;
00078     struct minmea_sentence_gga parsed_gga;
00079     struct minmea_sentence_gsa parsed_gsa;
```

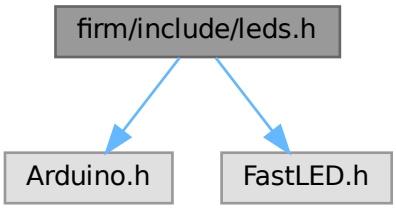
```
    struct minmea_sentence_gll parsed_gll;
    struct minmea_sentence_gst parsed_gst;
    struct minmea_sentence_gsv parsed_gsv;
    struct minmea_sentence_vtg parsed_vtg;
    struct minmea_sentence_zda parsed_zda;
};

extern uint32_t lastUiGpsMs;
extern volatile bool gpsDirty;
/* --- Prototipi Funzioni --- */

bool gpsInit();

bool nmea_gps_parse(String* nmea_message, struct parsed_nmea* nmea_ptr);

bool gpsAcquire(enum minmea_sentence_id sentence_type, struct parsed_nmea* nmea_ptr);

bool GetDate_and_Time(struct parsed_nmea* nmea_ptr);

bool GetPosition_and_Satellites(struct parsed_nmea* nmea_ptr);

bool GPS_sync(struct parsed_nmea* nmea_ptr, bool force);

bool save_gps_last(struct parsed_nmea* nmea_ptr);

bool save_gps_log20(struct parsed_nmea* nmea_ptr);

void check_GPS_sync_req(queue_t* recv_queue, queue_t* send_queue, struct parsed_nmea* nmea_ptr);

/* --- Funzioni di Debug --- */
void print_NMEA_rmc(void* frame);
void print_NMEA_gga(void* frame);
void print_NMEA_gst(void* frame);
void print_NMEA_gsv(void* frame);
void print_NMEA_vtg(void* frame);
void print_NMEA_zda(void* frame);

#endif  // GPS_H
```

5.11 firm/include/leds.h File Reference

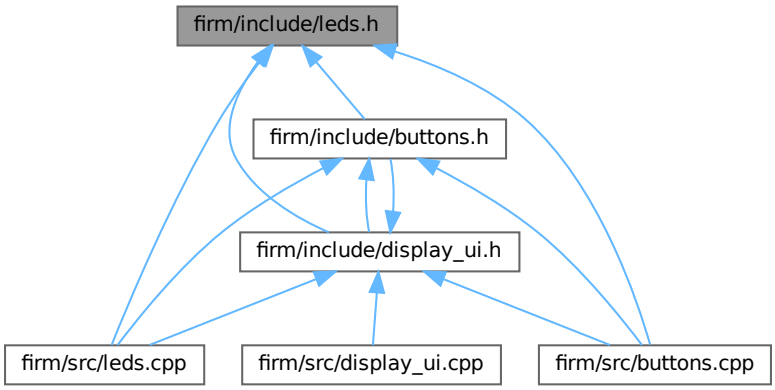
Definizioni hardware e prototipi per la gestione dei LED RGB.

```
#include <Arduino.h>
#include <FastLED.h>
```

Include dependency graph for leds.h:



This graph shows which files directly or indirectly include this file:



Macros

Configurazione Striscia LED

- #define LED_PIN 13
- #define NUM_LEDS 8
- #define LED_TYPE WS2812B
- #define COLOR_ORDER GRB

Functions

- void ledsInit ()

  Inizializza il controller FastLED con i parametri definiti nelle macro.
- void ledsShowInitAnimation ()

  Esegue una sequenza di accensione e spegnimento iniziale (Pink fade).
- void ledsNavigationAnimation ()

  Esegue un lampeggio Blu/Rosso per confermare lo spostamento nel menu.
- void ledsToggleAnimation ()

  Esegue un flash Verde per confermare una selezione o un cambio stato.
- void ledsInactivityAnimation ()

  Esegue un flash Giallo per segnalare il ritorno alla Home per inattività.

5.11.1 Detailed Description

Definizioni hardware e prototipi per la gestione dei LED RGB.

- Questo header contiene le macro di configurazione per la striscia LED (WS2812B) e i prototipi delle funzioni di animazione utilizzate per il feedback utente. La gestione fisica dei LED è affidata alla libreria FastLED.

Definition in file leds.h.

5.11.2 Macro Definition Documentation

5.11.2.1 COLOR_ORDER

```
#define COLOR_ORDER  GRB
```

Ordine dei canali colore (Green-Red-Blue) per il chipset \ utilizzato.
Definition at line 24 of file leds.h.

5.11.2.2 LED_PIN

```
#define LED_PIN  13
```

Pin GPIO dedicato alla trasmissione dati (Data Pin). \
Definition at line 19 of file leds.h.

5.11.2.3 LED_TYPE

```
#define LED_TYPE WS2812B
```

Tipo di controller integrato nei LED (es. WS2812B, SK6812).
Definition at line 21 of file leds.h.

5.11.2.4 NUM_LEDS

```
#define NUM_LEDS 8
```

Numero totale di LED presenti nella striscia.
Definition at line 20 of file leds.h.

5.11.3 Function Documentation

5.11.3.1 ledsInactivityAnimation()

```
void ledsInactivityAnimation ( )
```

Esegue un flash Giallo per segnalare il ritorno alla Home per inattività.
< Numero totale di LED presenti nella striscia.
Definition at line 64 of file leds.cpp.

5.11.3.2 ledsInit()

```
void ledsInit ( )
```

Inizializza il controller FastLED con i parametri definiti nelle macro.
< Tipo di controller integrato nei LED (es. WS2812B, SK6812).
< Pin GPIO dedicato alla trasmissione dati (Data Pin).

< Ordine dei canali colore (Green-Red-Blue) per il chipset utilizzato.
< Numero totale di LED presenti nella striscia.
Definition at line 8 of file leds.cpp.

5.11.3.3 ledsNavigationAnimation()

```
void ledsNavigationAnimation ( )
```

Esegue un lampeggio Blu/Rosso per confermare lo spostamento nel menu.
Definition at line 27 of file leds.cpp.

5.11.3.4 ledsShowInitAnimation()

```
void ledsShowInitAnimation ( )
```

Esegue una sequenza di accensione e spegnimento iniziale (Pink fade).
< Numero totale di LED presenti nella striscia.
< Numero totale di LED presenti nella striscia.
Definition at line 13 of file leds.cpp.

5.11.3.5 ledsToggleAnimation()

```
void ledsToggleAnimation ( )
```

Esegue un flash Verde per confermare una selezione o un cambio stato.
< Numero totale di LED presenti nella striscia.
Definition at line 53 of file leds.cpp.

5.12 leds.h

Go to the documentation of this file.

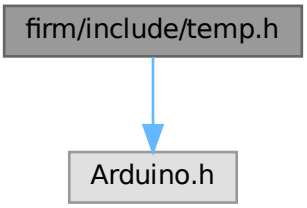
```
#ifndef LEDS_H
#define LEDS_H

#include <Arduino.h>
#include <FastLED.h>

#define LED_PIN                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                
    13
#define NUM_LEDS 8
#define LED_TYPE WS2812B
#define COLOR_ORDER                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                
    GRB
void ledsInit();

void ledsShowInitAnimation();

void ledsNavigationAnimation();

void ledsToggleAnimation();

void ledsInactivityAnimation();

#endif  // LEDS_H
```

5.13 firm/include/temp.h File Reference

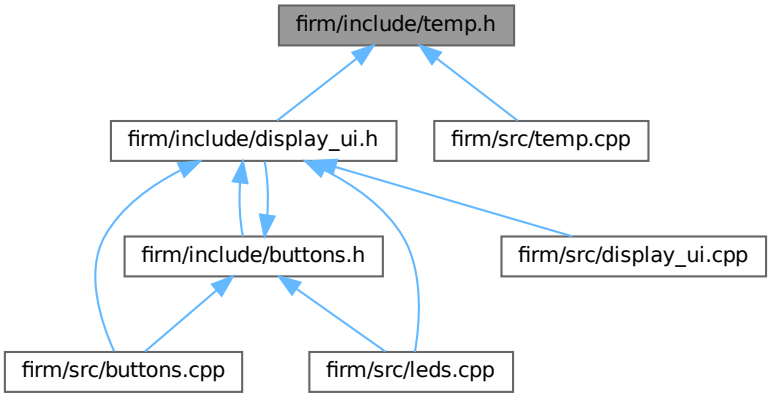
Definizioni hardware e prototipi per la lettura della temperatura.

`#include <Arduino.h>`

Include dependency graph for temp.h:



This graph shows which files directly or indirectly include this file:



Macros

Configurazione Hardware ADC

- #define TEMP_PIN 26
- #define ADC_BITS 12
- #define ADC_MAX_VAL 4095

Functions

- void TempInit ()

  Inizializza il pin del sensore e configura i parametri dell'ADC.
- float readTemp ()

  Esegue la lettura analogica e restituisce il valore in gradi Celsius.

5.13.1 Detailed Description

Definizioni hardware e prototipi per la lettura della temperatura.

- Questo header definisce i parametri di configurazione per l'ADC dell'RP2040 e i prototipi delle funzioni necessarie per l'acquisizione termica tramite sensore analogico lineare (es. TC1047A o LM35).

Definition in file temp.h.

5.13.2 Macro Definition Documentation

5.13.2.1 ADC_BITS

```
#define ADC_BITS 12
```

Risoluzione dei bit dell'ADC (12-bit per RP2040).
Definition at line 17 of file temp.h.

5.13.2.2 ADC_MAX_VAL

```
#define ADC_MAX_VAL 4095
```

Valore massimo dell'ADC calcolato come (2^ADC_BITS) - 1.
Definition at line 18 of file temp.h.

5.13.2.3 TEMP_PIN

`#define TEMP_PIN 26`
Pin GPIO (ADC0) collegato all'uscita del sensore di temperatura.
Definition at line 16 of file temp.h.

5.13.3 Function Documentation

5.13.3.1 readTemp()

`float readTemp ( )`
Esegue la lettura analogica e restituisce il valore in gradi Celsius.

- Acquisisce il valore di tensione dal pin TEMP_PIN e applica la formula di conversione lineare specifica per il sensore utilizzato.

  Returns

- float La temperatura corrente espressa in °C.

Formula di conversione per TC1047: Vout = 500mV a 0°C, con un coefficiente di 10mV/°C. Temp = (Vout - 0.5) / 0.01
Definition at line 16 of file temp.cpp.

5.13.3.2 TempInit()

`void TempInit ( )`
Inizializza il pin del sensore e configura i parametri dell'ADC.

- Prepara il GPIO selezionato come ingresso analogico e imposta la risoluzione corretta per le letture successive.

Definition at line 8 of file temp.cpp.

5.14 temp.h

Go to the documentation of this file.

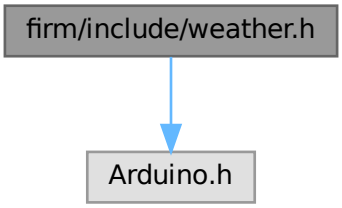
```
00001
00009 #ifndef TEMP_H
00010 #define TEMP_H
00011
00012 #include <Arduino.h>
00013
00016 #define TEMP_PIN    26
00017 #define ADC_BITS    12
00018 #define ADC_MAX_VAL 4095
00021 /* --- Funzioni di Interfaccia --- */
00022
00028 void TempInit();
00029
00036 float readTemp();
00037
00038 #endif  // TEMP_H
```

5.15 firm/include/weather.h File Reference

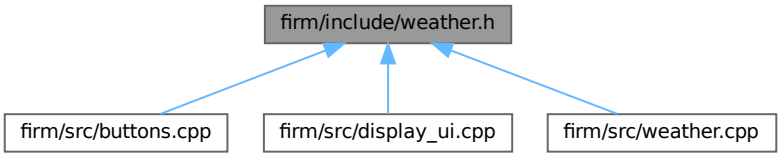
Definizioni della struttura dati e prototipi per il modulo Meteo.

```
#include <Arduino.h>
```

Include dependency graph for weather.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct WeatherState

  Contenitore globale per lo stato meteorologico.

Functions

- void Weather_Init ()

  Inizializza la struttura WeatherState con valori nulli o non validi.
- void Weather_HandleLine (const String &line)

  Elabora una riga di protocollo WX (WXC o WXD) per aggiornare lo stato.
- const char ∗ Weather_WeekdayName (uint8_t weekday)

  Converte un indice (0-6) nel nome abbreviato del giorno (es. "Lun").
- const char ∗ Weather_CodeToShortText (int code)

  Traduce un codice numerico WMO in una stringa breve leggibile (es. "Rain").

Variables

- WeatherState wx

  Istanza esterna della struttura meteo, accessibile da altri moduli (es. UI).

5.15.1 Detailed Description

Definizioni della struttura dati e prototipi per il modulo Meteo.

- Questo header definisce la struttura WeatherState, che funge da database locale per le informazioni meteorologiche correnti e le previsioni a breve termine (7 giorni).

Definition in file weather.h.

5.15.2 Function Documentation

5.15.2.1 Weather_CodeToShortText()

```
const char * Weather_CodeToShortText (
            int code )
```

Traduce un codice numerico WMO in una stringa breve leggibile (es. "Rain").

Parameters

code	Codice meteorologico standard.

Returns

const char∗ Descrizione testuale breve.

Definition at line 101 of file weather.cpp.

5.15.2.2 Weather_HandleLine()

```
void Weather_HandleLine (
            const String & line )
```

Elabora una riga di protocollo WX (WXC o WXD) per aggiornare lo stato.

Parameters

line	Stringa formattata CSV proveniente dal modulo di comunicazione.

Definition at line 31 of file weather.cpp.

5.15.2.3 Weather_Init()

```
void Weather_Init ( )
```

Inizializza la struttura WeatherState con valori nulli o non validi.
Definition at line 8 of file weather.cpp.

5.15.2.4 Weather_WeekdayName()

```
const char * Weather_WeekdayName (
            uint8_t weekday )
```

Converte un indice (0-6) nel nome abbreviato del giorno (es. "Lun").

Parameters

weekday	Indice del giorno.

Returns

const char* Stringa del nome del giorno.

Definition at line 96 of file weather.cpp.

5.15.3 Variable Documentation

5.15.3.1 wx

```
WeatherState wx  [extern]
```

Istanza esterna della struttura meteo, accessibile da altri moduli (es. UI).
Definition at line 6 of file weather.cpp.

5.16 weather.h

Go to the documentation of this file.

```
00001
00009 #ifndef WEATHER_H
00010 #define WEATHER_H
00011 #pragma once
00012
00013 #include <Arduino.h>
00014
00022 struct WeatherState {
00023     bool curValid = false;
00025     bool fcValid  = false;
00026     char city[24] = "NA";
00028     float curTempC   = NAN;
00029     float curWindKmh = NAN;
00030     int curHumidity  = -1;
00031     int curWcode     = -1;
00035     float tempC[7][24];
00036     uint8_t wcode[7][24];
00039     uint8_t weekday0 = 0;
00040     uint8_t recvMask = 0;
00043     uint32_t lastUpdateMs = 0;
00044 };
00045
00048 extern WeatherState wx;
00049
00050 /* --- Funzioni di Gestione --- */
00051
00056 void Weather_Init();
00057
00062 void Weather_HandleLine(const String& line);
00063
00064 /* --- Funzioni di Supporto per UI --- */
00065
00071 const char* Weather_WeekdayName(uint8_t weekday);
00072
00079 const char* Weather_CodeToShortText(int code);
00080
00081 #endif  // WEATHER_H
```

5.17 buttons.cpp

```
00001 #include "buttons.h"
00002
00003 #include "display_ui.h"
00004 #include "gps.h"
00005 #include "leds.h"
00006 #include "weather.h"
00007
00008 int menuIndex       = 0;
00009 bool menuOpen       = false;
00010 bool inSubmenu      = false;
00011 bool ambientDisplay = true;
00013 volatile bool up_press    = false;
00014 volatile bool down_press  = false;
00015 volatile bool left_press  = false;
00016 volatile bool right_press = false;
00018 uint32_t tLeft  = 0;
00019 uint32_t tRight = 0;
00020 const uint32_t COMBO_MS =
00021     300;
00022 uint32_t lastActivity = 0;
00023 const uint32_t DISPLAY_TIMEOUT_MS =
```

```
    5000000;
const uint32_t DEBOUNCE_DELAY = 40;
const int MENU_LENGTH = 5;
int ledsNumber        = 0;
enum Submenu : uint8_t {
    SUB_NONE,
    SUB_SETTINGS,
    SUB_METEO,
    SUB_GPS,
    SUB_SYSTEM,
    SUB_INFO
};

long long debounce_callback(alarm_id_t id, void* user_data);
void gpio_callback(uint gpio, uint32_t events);

void gpio_callback(uint gpio, uint32_t events) {
    // if released re apply the fall edge irq and disable the rising one
    if (events & GPIO_IRQ_EDGE_RISE) {
        gpio_set_irq_enabled(gpio, GPIO_IRQ_EDGE_RISE, false);
        gpio_set_irq_enabled(gpio, GPIO_IRQ_EDGE_FALL, true);
        return;
    }

    switch (gpio) {
        case SW_UP:
        case SW_DOWN:
        case SW_LEFT:
        case SW_RIGHT: {
            // Disables interrupt for the gpio
            gpio_set_irq_enabled(gpio, GPIO_IRQ_EDGE_FALL, false);
            // Add an alarm callback to be called after a delay specified in
            // milliseconds.
            add_alarm_in_ms(DEBOUNCE_DELAY, &debounce_callback, (void*)gpio, true);
        } break;

        default: {
            ;
        }
    }
    return;
}

long long debounce_callback(alarm_id_t id, void* user_data) {
    uint PinNum = (uint)user_data;

    if (!gpio_get(PinNum)) {
        Serial.print("Button press detected on pin ");
        Serial.println(PinNum);
        switch (PinNum) {
            case SW_UP:
                up_press = true;
                ;
                break;

            case SW_DOWN:
                down_press = true;
                ;
                break;

            case SW_LEFT:
                left_press = true;
                ;
                break;

            case SW_RIGHT:
                right_press = true;
                ;
                break;

            default: {
#if DEBUG == 1
                Serial.println("ERROR in debounce_callback: PinNum not correct");
#endif
                break;
            }
        }
        // re enables gpio interrupt but for complementary edge
        gpio_set_irq_enabled(PinNum, GPIO_IRQ_EDGE_RISE, true);
        return 0;
    }
    gpio_set_irq_enabled(PinNum, GPIO_IRQ_EDGE_FALL, true);
    return 0;
}

void buttonsInit() {
    const uint8_t SWpin[] = {SW_UP, SW_DOWN, SW_LEFT, SW_RIGHT};
```

```

#if DEBUG == 1
    Serial.println(F("Buttons and relative interrupts initialized"));
#endif

    for (uint8_t i = 0; i < (sizeof(SWpin) / sizeof(SWpin[0])); ++i) {
        pinMode(SWpin[i], INPUT_PULLUP);
    }
    // Imposta interrupt sui pulsanti
    gpio_set_irq_enabled_with_callback(SW_UP, GPIO_IRQ_EDGE_FALL | GPIO_IRQ_EDGE_RISE, true,
                                       &gpio_callback);
    for (uint8_t i = 1; i < (sizeof(SWpin) / sizeof(SWpin[0])); ++i) {
        gpio_set_irq_enabled(SWpin[i], GPIO_IRQ_EDGE_FALL | GPIO_IRQ_EDGE_RISE, true);
    }

    return;
}

void buttonsUpdate() {
    static Submenu currentSubmenu = SUB_NONE;

    KeyEvent ev = KEY_NONE;

    if (right_press)
        ev = KEY_RIGHT;
    else if (left_press)
        ev = KEY_LEFT;
    else if (up_press)
        ev = KEY_UP;
    else if (down_press)
        ev = KEY_DOWN;

    bool anypress = (ev != KEY_NONE);

    if (anypress) {
        // mapping per animazione LED
        if (ev == KEY_RIGHT)
            ledsNumber = 1;
        else if (ev == KEY_LEFT)
            ledsNumber = 5;
        else if (ev == KEY_UP)
            ledsNumber = 7;
        else if (ev == KEY_DOWN)
            ledsNumber = 3;

        ledsNavigationAnimation();

        // consuma i flag (così non si ripete)
        up_press = down_press = left_press = right_press = false;

        lastActivity = millis();
    }

    if (lastActivity == 0) {
        lastActivity = millis();
    }
    // Se ev è LEFT o RIGHT, gestisci combo/attesa
    if (ev == KEY_LEFT || ev == KEY_RIGHT) {
        if (checkLeftRightCombo(ev))
            return;  // se combo: toggle e fine
    }

    // se ritorna false => nessuna combo: puoi eseguire l'azione singola (LEFT o
    // RIGHT)

    if (!ambientDisplay && (millis() - lastActivity > DISPLAY_TIMEOUT_MS)) {
        ambientDisplay = true;
        menuOpen       = false;
        inSubmenu      = false;
        // ledsInactivityAnimation();
        drawHomeScreen();
        return;
    }

    // --- Se siamo in un ambiente display ---
    if (ambientDisplay && anypress) {
        ambientDisplay = false;
        menuOpen       = true;
        drawMenu(menuIndex);

        return;
    }

    // (NOTA: in questo punto left_press/right_press sono già stati consumati)*/

    // --- Menu aperto ---
    if (ev == KEY_UP && menuOpen) {  // scroll up
```

```
        menuIndex = (menuIndex == 0) ? MENU_LENGTH - 1 : menuIndex - 1;
        drawMenu(menuIndex);
        return;
    }

    if (ev == KEY_DOWN && menuOpen) {  // scroll down
        menuIndex = (menuIndex + 1) % MENU_LENGTH;
        drawMenu(menuIndex);
        return;
    }

    if (ev == KEY_RIGHT && menuOpen) {  // entra nel submenu
        inSubmenu = true;
        menuOpen  = false;
        switch (menuIndex) {
            case 0:
                drawSettingsScreen();
                currentSubmenu = SUB_SETTINGS;
                break;
            case 1:
                drawMeteoScreen();
                currentSubmenu = SUB_METEO;
                break;
            case 2:
                currentSubmenu = SUB_GPS;
                drawGPSScreen();  // disegna subito
                break;
                break;
            case 3:
                drawSystemScreen();
                currentSubmenu = SUB_SYSTEM;
                break;
            case 4:
                drawInfoScreen();
                currentSubmenu = SUB_INFO;
                break;
        }
    }
    if (currentSubmenu == SUB_METEO) {
        switch (ev) {
            case KEY_LEFT:
                gpsDayIndex = (gpsDayIndex + 6) % 7;
                drawMeteoScreen();
                return;
            case KEY_RIGHT:
                gpsDayIndex = (gpsDayIndex + 1) % 7;
                drawMeteoScreen();
                return;
            case KEY_UP:
                gpsHourIndex = (gpsHourIndex + 23) % 24;
                drawMeteoScreen();
                return;
            case KEY_DOWN:
                gpsHourIndex = (gpsHourIndex + 1) % 24;
                drawMeteoScreen();
                return;
            default:
                break;
        }
    }

    if (currentSubmenu == SUB_GPS && (ev == KEY_RIGHT)) {
        // Save GPS data to log file
        save_gps_log20(&gpsData);
        save_gps_last(&gpsData);
        drawGPSScreen();  // aggiorna schermata dopo salvataggio
    }

    if (currentSubmenu == SUB_SETTINGS) {
        if (ev == KEY_RIGHT) {
            // Toggle menu style
            ScreenStyle = (ScreenStyle == 0) ? 1 : 0;
            drawSettingsScreen();
            return;
        }

        if (ev == KEY_LEFT) {
            inSubmenu = false;
            menuOpen  = true;
            drawMenu(menuIndex);
            return;
        }
        return;
    }

    if (inSubmenu && currentSubmenu == SUB_GPS) {
        int timer               = 500;  // ms
```

```
        unsigned int lastMillis = millis();
        if (millis() - lastMillis > timer) {
            lastMillis = millis();
            drawGPSScreen();  // disegna subito
        }
    }

    if (inSubmenu) {
        if (ev == KEY_LEFT) {  // esci dal submenu
            inSubmenu = false;
            menuOpen  = true;
            drawMenu(menuIndex);
        }
        return;
    }

    /*
    if (left_press && menuOpen) {          // torni al menu (resta aperto)
    left_press = false;
    menuOpen = false;
        ambientDisplay = true;
        drawHomeScreen();
    }*/

    /*
    if (inSubmenu && currentSubmenu == SUB_GPS) {
     if (ev == KEY_UP) {
         gpsDayIndex = (gpsDayIndex == 0) ? 6 : gpsDayIndex - 1;
         drawGPSScreen();
         return;
     }
     if (ev == KEY_DOWN) {
         gpsDayIndex = (gpsDayIndex + 1) % 7;
         drawGPSScreen();
         return;
     }
     if (ev == KEY_LEFT) {
         gpsHourIndex = (gpsHourIndex == 0) ? 23 : gpsHourIndex - 1;
         drawGPSScreen();
         return;
     }
     if (ev == KEY_RIGHT) {
         gpsHourIndex = (gpsHourIndex + 1) % 24;
         drawGPSScreen();
         return;
     }
     */

    // uscita dal submenu (se vuoi usare una combo o long press)
    // esempio: LEFT+RIGHT combo già ce l'hai per toggle home/menu.
    return;
}

bool checkLeftRightCombo(KeyEvent ev) {
    static uint32_t lastL = 0;
    static uint32_t lastR = 0;

    uint32_t now = millis();

    if (ev == KEY_LEFT)
        lastL = now;
    if (ev == KEY_RIGHT)
        lastR = now;

    if (lastL && lastR && (uint32_t)abs((int32_t)(lastL - lastR)) <= COMBO_MS) {
        lastL = 0;
        lastR = 0;
        onLeftRightCombo();
        return true;  // combo gestita
    }

    // pulizia (opzionale)
    if (lastL && (now - lastL > COMBO_MS))
        lastL = 0;
    if (lastR && (now - lastR > COMBO_MS))
        lastR = 0;

    return false;
}

void onLeftRightCombo() {
    ledsToggleAnimation();
    ambientDisplay = !ambientDisplay;
    if (!ambientDisplay) {
        ambientDisplay = false;
        drawMenu(menuIndex);
    } else {
```

```
        ambientDisplay = true;
        drawHomeScreen();
    }
}
```

5.18 buzzer.cpp

```
#include "buzzer.h"

static uint8_t buzzerPin;

void buzzerInit(uint8_t pin) {
    buzzerPin = pin;
    pinMode(buzzerPin, OUTPUT);

#if DEBUG
    Serial.println("Buzzer initialized");
#endif

// Se la macro è definita a 1, fa il beep, altrimenti tace
#if BUZZER_INIT_BEEP == 1
    buzzerBeep();
#if DEBUG
    Serial.println("Buzzer init beep sent");
#endif
#endif
}

void buzzerBeep(uint16_t freq, uint16_t duration) {
    tone(buzzerPin, freq);
    delay(duration);
    noTone(buzzerPin);
}
```

5.19 display_ui.cpp

```
#include "display_ui.h"

#include "gps.h"
#include "weather.h"

extern volatile bool gpsDirty;
extern struct parsed_nmea
    gpsData;
static uint32_t lastGpsUiMs  = 0;
uint8_t gpsDayIndex          = 0;
uint8_t gpsHourIndex         = 0;
static uint8_t settingsIndex = 0;
uint8_t ScreenStyle          = 1;
const char* menuItems[] = {"Settings", "Meteo", "GPS", "System", "Info"};

const int menuLength = sizeof(menuItems) / sizeof(menuItems[0]);

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

bool displayInit() {
    Wire.setSDA(DISPLAY_SDA_I2C_PIN);
    Wire.setSCL(DISPLAY_SCL_I2C_PIN);
    Wire.begin();
    if (!display.begin(SSD1306_SWITCHCAPVCC, SCREEN_ADDRESS)) {
        Serial.println(F("SSD1306 allocation failed"));
        return false;  // Exit if display initialization fails
    }

    display.clearDisplay();
    display.setTextColor(SSD1306_WHITE);
    display.setTextSize(1);
    display.setCursor(0, 0);
    display.display();
    return true;
}

static void print2d(uint8_t v) {
    if (v < 10)
        display.print("0");
    display.print(v);
}

static inline bool dayHasForecast(uint8_t d) {
    return (wx.recvMask & (1u << (d))) != 0;
}

```

```
00050 void drawHomeScreen() {
00051     display.clearDisplay();
00052     display.setTextColor(SSD1306_WHITE);
00053     display.setTextSize(1);
00054     display.setCursor(0, 0);
00055     display.println("Weather:");
00056     display.drawLine(0, 10, 127, 10, SSD1306_WHITE);
00057     display.setCursor(0, 14);
00058     display.print("City: ");
00059     display.print(wx.curValid ? wx.city : "NA");
00060     display.setCursor(0, 24);
00061     if (wx.curValid) {
00062         display.print(wx.curTempC, 1);
00063         display.print("C ");
00064         display.print(Weather_CodeToShortText(wx.curWcode));
00065
00066     } else {
00067         display.print("NA");
00068     }
00069     display.setCursor(0, 34);
00070     display.print("Hum: ");
00071     if (!wx.curValid || isnan(wx.curHumidity))
00072         display.print("--");
00073     else {
00074         display.print((int)roundf(wx.curHumidity));
00075         display.print("%");
00076     }
00077     display.setCursor(0, 44);
00078     struct minmea_sentence_rmc rmc = gpsData.parsed_rmc;
00079     struct minmea_sentence_gga gga = gpsData.parsed_gga;
00080     display.print("Time: ");
00081     display.print(gga.time.hours);
00082     display.print(":");
00083     display.print(gga.time.minutes);
00084     display.print(":");
00085     display.print(gga.time.seconds);
00086     display.setCursor(0, 54);
00087     display.print("Date: ");
00088     display.print(rmc.date.day);
00089     display.print("/");
00090     display.print(rmc.date.month);
00091     display.print("/");
00092     display.print(rmc.date.year);
00093
00094     display.display();
00095 }
00096
00097 void drawMenu(int index) {
00098     if (ScreenStyle == 1) {
00099         display.clearDisplay();
00100         display.setTextColor(SSD1306_WHITE);
00101         display.setCursor(0, 0);
00102         display.println("Menu:");
00103         for (int i = 0; i < menuLength; i++) {
00104             if (i == index) {
00105                 display.print("> ");
00106             } else {
00107                 display.print("  ");
00108             }
00109             display.println(menuItems[i]);
00110         }
00111         display.display();
00112         return;
00113     } else {
00114         drawMenu_evi(index);
00115     }
00116 }
00117
00118 void drawMenu_evi(int index) {
00119     display.clearDisplay();
00120     display.setTextSize(1);
00121
00122     // Titolo
00123     display.setTextColor(SSD1306_WHITE);
00124     display.setCursor(0, 0);
00125     display.print("Menu");
00126     display.drawLine(0, 10, 127, 10, SSD1306_WHITE);
00127
00128     // Layout
00129     const int firstY = 14;  // inizio lista
00130     const int lineH  = 10;  // altezza riga (8px font + spazio)
00131     const int boxH   = 9;   // altezza box evidenziato
00132
00133     for (int i = 0; i < menuLength; i++) {
00134         int y = firstY + i * lineH;
00135
00136         if (i == index) {
```

```
            // rettangolo evidenziato
            display.fillRect(0, y - 1, 128, boxH, SSD1306_WHITE);
            display.setTextColor(SSD1306_BLACK);  // testo nero sul bianco
        } else {
            display.setTextColor(SSD1306_WHITE);
        }

        display.setCursor(2, y);
        display.print(menuItems[i]);
    }

    display.display();
}

void drawSettingsScreen() {
    display.clearDisplay();
    display.setTextSize(1);
    display.setTextColor(SSD1306_WHITE);

    display.setCursor(0, 0);
    display.print("Settings");
    display.drawLine(0, 10, 127, 10, SSD1306_WHITE);

    display.setCursor(0, 16);
    display.print(settingsIndex == 0 ? "> " : "  ");
    display.print("Menu style: ");
    display.print(ScreenStyle == 1 ? "Simple" : "Highlight");

    display.setCursor(0, 54);
    display.print("R=Change  L=Back");
    display.display();
}

void drawMeteoScreen() {
    display.clearDisplay();
    display.setTextSize(1);
    display.setTextColor(SSD1306_WHITE);

    // Titolo
    display.setCursor(0, 0);
    display.print("Meteo:");
    display.drawLine(0, 10, 127, 10, SSD1306_WHITE);
    display.setCursor(0, 14);

    // Citta+giorno
    display.print("City:");
    display.print(wx.curValid ? wx.city : "NA");
    display.print(" ");
    uint8_t wd = (wx.weekday0 + gpsDayIndex) % 7;
    display.print(Weather_WeekdayName(wd));

    // Riga: giorno selezionato + ora selezionata
    display.setCursor(0, 24);
    display.print("g");
    display.print(gpsDayIndex);
    display.print(" h");
    display.print(gpsHourIndex);

    display.setCursor(0, 34);

    if (dayHasForecast(gpsDayIndex)) {
        float t      = wx.tempC[gpsDayIndex][gpsHourIndex];
        uint8_t code = wx.wcode[gpsDayIndex][gpsHourIndex];

        display.print("Fc: ");
        if (isnan(t))
            display.print("--");
        else {
            display.print(t, 1);
            display.print("C ");
        }

        if (code == 255)
            display.print("--");
        else
            display.print(Weather_CodeToShortText(code));
    } else {
        display.print("Fc: no data");
    }

    // Riga 4: Now (corrente) + Humidita corrente
    display.setCursor(0, 46);
    display.print("Now: ");
    if (wx.curValid) {
        display.print(wx.curTempC, 1);
        display.print("C ");
        display.print(Weather_CodeToShortText(wx.curWcode));
```

```
    } else {
        display.print("NA");
    }

    display.setCursor(0, 56);
    display.print("Hum: ");
    // usa il NOME che hai realmente in WeatherState:
    // se lo hai chiamato curHumPct:
    if (!wx.curValid || isnan(wx.curHumidity))
        display.print("--");
    else {
        display.print((int)roundf(wx.curHumidity));
        display.print("%");
    }

    display.display();

    Serial.println(gpsDayIndex);
    Serial.println(gpsHourIndex);
}

void drawGPSScreen() {
    // normIndices();

    display.clearDisplay();
    display.setTextSize(1);
    display.setTextColor(SSD1306_WHITE);

    display.setCursor(0, 0);
    display.print("GPS & Meteo");
    display.setCursor(50, 0);
    display.print("R = Save data to log");
    display.drawLine(0, 10, 127, 10, SSD1306_WHITE);

    struct minmea_sentence_rmc rmc = gpsData.parsed_rmc;
    struct minmea_sentence_gga gga = gpsData.parsed_gga;

    bool fix = gga.fix_quality > 0;

    float lat = minmea_tocoord(&gga.latitude);
    float lon = minmea_tocoord(&gga.longitude);

    // Riga: Lat
    display.setCursor(0, 14);
    display.print("Lat:");
    if (fix)
        display.print(lat, 5);
    else
        display.print("NA");

    // Riga: Lon
    display.setCursor(0, 24);
    display.print("Lon:");
    if (fix)
        display.print(lon, 5);
    else
        display.print("NA");

    // Riga: Fix + sats
    display.setCursor(0, 34);
    display.print("FixQ:");
    display.print(gga.fix_quality);
    display.print(" Sat:");
    display.print(gga.satellites_tracked);

    // Riga: Meteo (ora selezionata)
    display.setCursor(0, 44);
    display.print("Time: ");
    display.print(gga.time.hours + 1);
    display.print(":");
    display.print(gga.time.minutes);
    display.print(":");
    display.print(gga.time.seconds);
    display.display();
    display.setCursor(0, 54);
    display.print("Date: ");
    display.print(rmc.date.day);
    display.print("/");
    display.print(rmc.date.month);
    display.print("/");
    display.print(rmc.date.year);
    display.display();
}

void drawSelected(const char* title) {
    display.clearDisplay();
    display.setTextColor(SSD1306_WHITE);
```

```
    display.setTextSize(1);
    display.setCursor(0, 0);
    display.print("Selected:");
    display.setCursor(0, 12);
    display.print(title);
    display.display();
}

void drawSystemScreen() {
    drawSelected("System");
}
void drawInfoScreen() {
    drawSelected("Info");
}

void updateDisplayTimeout() {
    if (menuOpen || inSubmenu) {
        if (millis() - lastActivity > DISPLAY_TIMEOUT_MS) {
            menuOpen       = false;
            inSubmenu      = false;
            ambientDisplay = true;
            drawHomeScreen();
        }
    }
}

void setBrightness(uint8_t level) {
    // Placeholder: Implement brightness control if hardware supports it
    display.ssd1306_command(SSD1306_SETCONTRAST);
    display.ssd1306_command(level);
    // level 0-255
}
```

5.20 filesystem.cpp

```
#include "filesystem.h"

volatile bool updated = false;

volatile bool driveConnected = false;

volatile bool inPrinting = false;

bool FatFS_Setup() {
    inPrinting = true;
    if (!FatFS.begin()) {
        Serial.println("FatFS first mount failed. Formatting...");
        FatFS.format();
        if (!FatFS.begin()) {
            Serial.println("FATAL: FatFS format/mount failed!");
            while (1) delay(10);
        }
    }
    inPrinting = false;
    Serial.println("Fat filesystem started");
    VFS.root(FatFS);  // Enables POSIX functions for I/O
    return true;
}

bool FatFSUSB_Setup() {
    // Set up callbacks
    FatFSUSB.onUnplug(unplug);
    FatFSUSB.onPlug(plug);
    FatFSUSB.driveReady(mountable);

    if (!FatFSUSB.begin()) {
        Serial.println("FATAL failed to start USB Mass Storage");
        while (1) delay(10);
    }
    delay(3000);
    if (Serial)
        Serial.println("USB Mass Storage started");

    return true;
}

void unplug(uint32_t i) {
    (void)i;
    driveConnected = false;
    updated        = true;
}

void plug(uint32_t i) {
    (void)i;
    driveConnected = true;
```

```
    FatFS.end();
}

bool mountable(uint32_t i) {
    (void)i;
    return !inPrinting;
}

bool sync_files() {
    bool ret = true;

    if (updated && !driveConnected) {
        Serial.println("\n\nDisconnected, new file listing:");
        FatFS.end();
        ret     = FatFS_Setup();
        updated = false;
    }
    return ret;
}
```

5.21 gps.cpp

```
#include "gps.h"

bool gpsInit() {
    Serial.println("=== GPS Init ===");

    // Led alive init
    pinMode(LED_ALIVE, OUTPUT);
    digitalWrite(LED_ALIVE, LOW);  // LED alive off
    Serial.print("Led alive pin: ");
    Serial.print(LED_ALIVE);
    Serial.println(" inizializzato come output.");

    // Enable GPS module
    pinMode(GPS_EN_PIN, OUTPUT);
    digitalWrite(GPS_EN_PIN, LOW);  // bring down GPS_EN signal to enable (pmos)
    Serial.print("GPS enable pin: ");
    Serial.print(GPS_EN_PIN);
    Serial.println(" alimentato.");

    // Enable input for Pulse signal from gps
    pinMode(GPS_PPS_PIN, INPUT);
    Serial.print("Gps PPS pin: ");
    Serial.print(GPS_PPS_PIN);
    Serial.println(" inizializzato come input.");

    // UART1 for the GPS. Serial2 maps to UART1 in Arduino framework!
    Serial2.setTX(UART1_TX_PIN);
    Serial2.setRX(UART1_RX_PIN);
    Serial2.begin(GPSBAUD);  // start uart1
    Serial.println("UART1 started on Serial2, listening for GPS data...");

    // Wait at least 10 sec for a GPS string.
    int i;
    for (i = 0; i < 1000 && (Serial2.available() == 0); ++i) delay(1);

    if (i == 1000) {
        Serial.println("ERROR: Canot start the GPS module");
        return false;
    }
    Serial.println("OK!");

    return true;
}

/*
void EmptyGpsBuffer(){
  while(Serial2.available()) {
    Serial2.read();  // Read and discard
  }
}
*/

bool nmea_gps_parse(String* nmea_message, struct parsed_nmea* nmea_ptr) {
    bool state = true;

    if (nmea_ptr == NULL) {
        Serial.println("ERROR in nmea_gps_parse: nmea struct is NULL");
        return false;
    }

    switch (minmea_sentence_id(nmea_message->c_str(), true)) {
        case MINMEA_SENTENCE_RMC: {
            struct minmea_sentence_rmc* frame = &(nmea_ptr->parsed_rmc);
```

```
            if (minmea_parse_rmc(frame, nmea_message->c_str()))
                print_NMEA_rmc(frame);
            else
                (state = false);
        } break;

        case MINMEA_SENTENCE_GGA: {
            struct minmea_sentence_gga* frame = &(nmea_ptr->parsed_gga);
            if (minmea_parse_gga(frame, nmea_message->c_str()))
                print_NMEA_gga(frame);
            else
                (state = false);
        } break;

        case MINMEA_SENTENCE_GST: {
            struct minmea_sentence_gst* frame = &(nmea_ptr->parsed_gst);
            if (minmea_parse_gst(frame, nmea_message->c_str()))
                print_NMEA_gst(frame);
            else
                (state = false);
        } break;

        case MINMEA_SENTENCE_GSV: {
            struct minmea_sentence_gsv* frame = &(nmea_ptr->parsed_gsv);
            if (minmea_parse_gsv(frame, nmea_message->c_str()))
                print_NMEA_gsv(frame);
            else
                (state = false);
        } break;

        case MINMEA_SENTENCE_VTG: {
            struct minmea_sentence_vtg* frame = &(nmea_ptr->parsed_vtg);
            if (minmea_parse_vtg(frame, nmea_message->c_str()))
                print_NMEA_vtg(frame);
            else
                (state = false);
        } break;

        case MINMEA_SENTENCE_ZDA: {
            struct minmea_sentence_zda* frame = &(nmea_ptr->parsed_zda);
            if (minmea_parse_zda(frame, nmea_message->c_str()))
                print_NMEA_zda(frame);
            else
                (state = false);
        } break;
        default: {
            (state = false);
        }
    }

    if (state)
        return true;
#if DEBUG
    Serial.print("WARNING: Could not parse this nmea string ");
    Serial.println(nmea_message->c_str());
#endif
    return false;
}

bool gpsAcquire(enum minmea_sentence_id sentence_type, struct parsed_nmea* nmea_ptr) {
    String nmea_message;

    digitalWrite(LED_ALIVE, HIGH);

    for (int i = 0; i < GPS_ACQUIRE_MAX_TRIES; ++i) {
        for (int c = 0; Serial2.available() == 0; ++c) delay(1);

        nmea_message = Serial2.readStringUntil('\n');
        digitalWrite(LED_ALIVE, LOW);

        if (minmea_sentence_id(nmea_message.c_str(), true) == sentence_type) {
            if (nmea_gps_parse(&nmea_message, nmea_ptr))
                return true;
        }
    }

#if DEBUG
    Serial.println(
        "WARNING in gpsAcquire: max tries reached before correctly "
        "acquiring the requested gps message\n");
#endif
    return false;
}

bool GetPosition_and_Satellites(struct parsed_nmea* nmea_ptr) {
    if (gpsAcquire(MINMEA_SENTENCE_GGA, nmea_ptr)) {
        return true;
```

```
    }
#if DEBUG
    Serial.println("WARNING in GetPosition_and_Satellites: failed to get data");
#endif
    return false;
}

bool GetDate_and_Time(struct parsed_nmea* nmea_ptr) {
    if (gpsAcquire(MINMEA_SENTENCE_RMC, nmea_ptr)) {
        return true;
    }
#if DEBUG
    Serial.println("WARNING in GetDate_and_Time: failed to get data");
#endif
    return false;
}

bool GPS_sync(struct parsed_nmea* nmea_ptr, bool force) {
    // retain value across function calls
    static unsigned long lastGPSsync = 0, lastGPSwrite = 0;

    if ((millis() - lastGPSsync > GPS_SYNC_TIMEOUT_MSEC) || force) {
        if (GetPosition_and_Satellites(nmea_ptr) && GetDate_and_Time(nmea_ptr)) {
            lastGPSsync = millis();
            if (((millis() - lastGPSwrite) > GPS_WRITE_FREQ) ||
                force)  // Write only at a certain frequency otherwise flash will wear
                        // out
            {
                if (save_gps_last(nmea_ptr))
                    lastGPSwrite = millis();
                else
                    return false;
            }
        } else  // It's only a warning so return true at the end
        {
#if DEBUG
            Serial.println("WARNING in GPS_sync: failed to get data");
#endif
        }
    }
    return true;
}

bool save_gps_last(struct parsed_nmea* nmea_ptr) {
    if (inPrinting || driveConnected || updated)
        return false;

    struct minmea_sentence_gga* gga = &(nmea_ptr->parsed_gga);
    if (gga->fix_quality <= 0)
        return false;
    char line[96];
    float lat = minmea_tocoord(&(gga->latitude));
    float lon = minmea_tocoord(&(gga->longitude));

    inPrinting = true;
    FILE* f    = fopen(GPS_LAST_PATH, "w");  // sovrascrive sempre
    if (!f) {
        inPrinting = false;
        return false;
    }
    int n = snprintf(line, sizeof(line), "%.6f,%.6f\n", lat, lon);
    fwrite(line, 1, (size_t)n, f);
    fclose(f);
    inPrinting = false;
    return true;
}

bool save_gps_log20(struct parsed_nmea* nmea_ptr) {
    if (inPrinting || driveConnected || updated)
        return false;

    struct minmea_sentence_rmc* rmc = &(nmea_ptr->parsed_rmc);
    if (!rmc->valid)
        return false;

    // nuova riga da aggiungere
    char newline[96];
    float lat = minmea_tocoord(&(rmc->latitude));
    float lon = minmea_tocoord(&(rmc->longitude));
    int newn  = snprintf(newline, sizeof(newline), "%.6f,%.6f\n", lat, lon);
    if (newn <= 0)
        return false;

    // buffer per massimo 20 righe
    char lines[GPS_MAX_LINES][96] = {{0}};
    int count                     = 0;

```

```
    inPrinting = true;

    // 1) leggi file esistente (se c'è)
    FILE* f = fopen(GPS_LOG_PATH, "r");
    if (f) {
        char tmp[96];
        while (fgets(tmp, sizeof(tmp), f)) {
            // copia la riga, limitando dimensione
            strncpy(lines[count % GPS_MAX_LINES], tmp, sizeof(lines[0]) - 1);
            lines[count % GPS_MAX_LINES][sizeof(lines[0]) - 1] = '\0';
            count++;
            if (count > 200)
                break;  // safety (non dovrebbe mai succedere)
        }
        fclose(f);
    }

    // 2) costruisci nuova lista: ultime 19 + nuova
    int start    = (count > GPS_MAX_LINES - 1) ? (count - (GPS_MAX_LINES - 1)) : 0;
    int outCount = 0;

    char out[GPS_MAX_LINES][96] = {{0}};
    for (int i = start; i < count; i++) {
        strncpy(out[outCount], lines[i % GPS_MAX_LINES], sizeof(out[0]) - 1);
        out[outCount][sizeof(out[0]) - 1] = '\0';
        outCount++;
    }

    // aggiungi nuova riga
    strncpy(out[outCount], newline, sizeof(out[0]) - 1);
    out[outCount][sizeof(out[0]) - 1] = '\0';
    outCount++;

    // 3) riscrivi file con esattamente outCount righe (max 20)
    f = fopen(GPS_LOG_PATH, "w");
    if (!f) {
        inPrinting = false;
        return false;
    }

    for (int i = 0; i < outCount; i++) {
        size_t n = strlen(out[i]);
        if (n)
            fwrite(out[i], 1, n, f);
    }
    fclose(f);

    inPrinting = false;
    return true;
}

void print_NMEA_rmc(void* frame) {
#if DEBUG
    struct minmea_sentence_rmc* rmc = (struct minmea_sentence_rmc*)frame;
    Serial.printf("$xxRMC: raw coordinates and speed: (%d/%d,%d/%d) %d/%d\n", rmc->latitude.value,
                  rmc->latitude.scale, rmc->longitude.value, rmc->longitude.scale, rmc->speed.value,
                  rmc->speed.scale);
    Serial.printf(
        "$xxRMC fixed-point coordinates and speed scaled to three "
        "decimal places: (%d,%d) %d\n",
        minmea_rescale(rmc->latitude, 1000), minmea_rescale(rmc->longitude, 1000),
        minmea_rescale(rmc->speed, 1000));
    Serial.printf("$xxRMC floating point degree coordinates and speed: (%f,%f) %f\n",
                  minmea_tocoord(rmc->latitude), minmea_tocoord(rmc->longitude),
                  minmea_tofloat(rmc->speed));
#endif
    return;
}

void print_NMEA_gga(void* frame) {
    struct minmea_sentence_gga* gga = (struct minmea_sentence_gga*)frame;
    Serial.printf("$xxGGA: fix quality: %d\n", gga->fix_quality);
    return;
}

void print_NMEA_gst(void* frame) {
#if DEBUG
    struct minmea_sentence_gst* gst = (struct minmea_sentence_gst*)frame;
    Serial.printf(
        "$xxGST: raw latitude,longitude and altitude error deviation: "
        "(%d/%d,%d/%d,%d/%d)\n",
        gst->latitude_error_deviation.value, gst->latitude_error_deviation.scale,
        gst->longitude_error_deviation.value, gst->longitude_error_deviation.scale,
        gst->altitude_error_deviation.value, gst->altitude_error_deviation.scale);
    Serial.printf(
        "$xxGST fixed point latitude,longitude and altitude error "
        "deviation scaled to one decimal place: (%d,%d,%d)\n",
```

```
        minmea_rescale(gst->latitude_error_deviation, 10),
        minmea_rescale(gst->longitude_error_deviation, 10),
        minmea_rescale(gst->altitude_error_deviation, 10));
    Serial.printf(
        "$xxGST floating point degree latitude, longitude and altitude "
        "error deviation: (%f,%f,%f)",
        minmea_tofloat(gst->latitude_error_deviation),
        minmea_tofloat(gst->longitude_error_deviation),
        minmea_tofloat(gst->altitude_error_deviation));
#endif
    return;
}

void print_NMEA_gsv(void* frame) {
#if DEBUG
    struct minmea_sentence_gsv* gsv = (struct minmea_sentence_gsv*)frame;
    Serial.printf("$xxGSV: message %d of %d\n", gsv->msg_nr, gsv->total_msgs);
    Serial.printf("$xxGSV: satellites in view: %d\n", gsv->total_sats);
    for (int i = 0; i < 4; i++) {
        Serial.printf("$xxGSV: sat nr %d, elevation: %d, azimuth: %d, snr: %d dbm\n",
                      gsv->sats[i].nr, gsv->sats[i].elevation, gsv->sats[i].azimuth,
                      gsv->sats[i].snr);
    }
#endif
    return;
}

void print_NMEA_zda(void* frame) {
#if DEBUG
    struct minmea_sentence_zda* zda = (struct minmea_sentence_zda*)frame;
    Serial.printf("$xxZDA: %d:%d:%d %02d.%02d.%d UTC%+03d:%02d\n", zda->time.hours,
                  zda->time.minutes, zda->time.seconds, zda->date.day, zda->date.month,
                  zda->date.year, zda->hour_offset, zda->minute_offset);
#endif
    return;
}

void print_NMEA_vtg(void* frame) {
#if DEBUG
    struct minmea_sentence_vtg* vtg = (struct minmea_sentence_vtg*)frame;
    Serial.printf("$xxVTG: true track degrees = %f\n", minmea_tofloat(vtg->true_track_degrees));
    Serial.printf("magnetic track degrees = %f\n", minmea_tofloat(vtg->magnetic_track_degrees));
    Serial.printf("speed knots = %f\n", minmea_tofloat(vtg->speed_knots));
    Serial.printf("speed kph = %f\n", minmea_tofloat(vtg->speed_kph));
#endif
    return;
}

void gpsSerialDebugLog() {
    static char nmea[MINMEA_MAX_SENTENCE_LENGTH];
    static uint8_t idx = 0;

    while (Serial2.available()) {
        char c = Serial2.read();

        if (c == '\n') {
            nmea[idx] = '\0';

            Serial.print("GPS RAW: ");
            Serial.println(nmea);

            idx = 0;  // reset buffer
        } else {
            if (idx < MINMEA_MAX_SENTENCE_LENGTH - 1) {
                nmea[idx++] = c;
            }
        }
    }
}

void check_GPS_sync_req(queue_t* recv_queue, queue_t* send_queue, struct parsed_nmea* nmea_ptr) {
    if (!queue_is_empty(recv_queue)) {
        uint8_t cmd;
        queue_remove_blocking(recv_queue, &cmd);  // read the item in the queue
        if (cmd == GPS_SYNC_REQ) {
            if (GPS_sync(nmea_ptr, true))
                exit(EXIT_FAILURE);
            cmd = GPS_SYNC_ACK;
            if (queue_try_add(send_queue, &cmd))
                ;
        }
    }
    return;
}
```

5.22 leds.cpp

```
#include "leds.h"

#include "buttons.h"
#include "display_ui.h"

CRGB leds[NUM_LEDS];

void ledsInit() {
    FastLED.addLeds<LED_TYPE, LED_PIN, COLOR_ORDER>(leds, NUM_LEDS);
    return;
}

void ledsShowInitAnimation() {
    for (int i = 0; i < NUM_LEDS; i++) {
        leds[i] = CRGB::LightPink;
        FastLED.show();
        delay(50);
    }
    delay(200);
    for (int i = 0; i < NUM_LEDS; i++) {
        leds[i] = CRGB::Black;
        FastLED.show();
        delay(50);
    }
}

void ledsNavigationAnimation() {
    FastLED.clear();
    leds[ledsNumber] = CRGB::Blue;

    // Gestione rollover per striscia da 8 LED
    if (ledsNumber + 1 == 8) {
        leds[0] = CRGB::Red;
    } else {
        leds[ledsNumber + 1] = CRGB::Red;
    }

    FastLED.show();
    delay(200);
    FastLED.clear();
    FastLED.show();

    // Reset stato LED
    leds[ledsNumber] = CRGB::Black;
    if (ledsNumber + 1 == 8) {
        leds[0] = CRGB::Black;
    } else {
        leds[ledsNumber + 1] = CRGB::Black;
    }
    FastLED.show();
}

void ledsToggleAnimation() {
    FastLED.clear();
    for (int i = 0; i < NUM_LEDS; i++) {
        leds[i] = CRGB::Green;
    }
    FastLED.show();
    delay(200);
    FastLED.clear();
    FastLED.show();
}

void ledsInactivityAnimation() {
    FastLED.clear();
    for (int i = 0; i < NUM_LEDS; i++) {
        leds[i] = CRGB::Yellow;
    }
    FastLED.show();
    delay(200);
    FastLED.clear();
    FastLED.show();
}
```

5.23 temp.cpp

```
#include "temp.h"

#include <Arduino.h>

#include "hardware/adc.h"
#include "pico/stdlib.h"
```

```

void TempInit() {
    // Imposta la risoluzione ADC a 12 bit (standard per RP2040)
    analogReadResolution(12);

    // Esegue una lettura a vuoto per stabilizzare l'ADC
    analogReadTemp();
}

float readTemp() {
    // Legge il valore grezzo dall'ADC (0 - 4095)
    uint16_t raw = analogReadTemp();

    // Conversione in tensione (Volt)
    float v = raw * 3.3f / 4095.0f;

    return (v - 0.5f) / 0.01f;
}
```

5.24 weather.cpp

```
#include "weather.h"

#include <math.h>
#include <string.h>

WeatherState wx;

void Weather_Init() {
    wx = {};
    strcpy(wx.city, "NA");
    for (int d = 0; d < 7; d++) {
        for (int h = 0; h < 24; h++) {
            wx.tempC[d][h] = NAN;
            wx.wcode[d][h] = 255;
        }
    }
}

static bool splitField(const String& s, int& pos, String& out) {
    int comma = s.indexOf(',', pos);
    if (comma < 0) {
        out = s.substring(pos);
        pos = s.length();
        return out.length() > 0;
    }
    out = s.substring(pos, comma);
    pos = comma + 1;
    return true;
}

void Weather_HandleLine(const String& line) {
    // Parsing dati correnti
    if (line.startsWith("WXC,")) {
        int pos = 4;
        String city, t, w, hum, code, wd;
        if (!splitField(line, pos, city))
            return;
        if (!splitField(line, pos, t))
            return;
        if (!splitField(line, pos, w))
            return;
        if (!splitField(line, pos, hum))
            return;
        if (!splitField(line, pos, code))
            return;
        if (!splitField(line, pos, wd))
            return;

        city.replace(",", "_");
        city.toCharArray(wx.city, sizeof(wx.city));
        wx.curTempC     = t.toFloat();
        wx.curWindKmh   = w.toFloat();
        wx.curHumidity  = hum.toInt();
        wx.curWcode     = code.toInt();
        wx.weekday0     = (uint8_t)wd.toInt();
        wx.curValid     = true;
        wx.lastUpdateMs = millis();
        return;
    }

    // Parsing previsioni giornaliere
    if (line.startsWith("WXD,")) {
        int pos = 4;
        String dStr;
```

```
        if (!splitField(line, pos, dStr))
            return;
        int d = dStr.toInt();
        if (d < 0 || d > 6)
            return;

        // Estrazione 24 temperature orarie
        for (int h = 0; h < 24; h++) {
            String v;
            if (!splitField(line, pos, v))
                return;
            wx.tempC[d][h] = v.toFloat();
        }
        // Estrazione 24 codici meteo orari (WMO codes)
        for (int h = 0; h < 24; h++) {
            String v;
            if (!splitField(line, pos, v))
                return;
            int code       = v.toInt();
            wx.wcode[d][h] = (uint8_t)constrain(code, 0, 255);
        }

        // Aggiorna maschera bitwise per verificare se tutti i 7 giorni sono stati
        // ricevuti
        wx.recvMask |= (1u << d);
        wx.fcValid      = (wx.recvMask == 0x7F);
        wx.lastUpdateMs = millis();
        return;
    }
}

const char* Weather_WeekdayName(uint8_t w) {
    static const char* names[7] = {"Lun", "Mar", "Mer", "Gio", "Ven", "Sab", "Dom"};
    return names[w % 7];
}

const char* Weather_CodeToShortText(int code) {
    if (code == 0)
        return "Clear";
    if (code >= 1 && code <= 3)
        return "Cloud";
    if (code == 45 || code == 48)
        return "Fog";
    if (code >= 51 && code <= 57)
        return "Drizz";
    if (code >= 61 && code <= 67)
        return "Rain";
    if (code >= 71 && code <= 77)
        return "Snow";
    if (code >= 80 && code <= 82)
        return "Shower";
    if (code >= 95)
        return "Storm";
    return "WX";
}
```